Traceability and estimate coverage as corpus-eligibility constraints: a probe of 38 Apache projects

Malek Khannoussi
independent researcher
khannoussimalek@gmail.com

August 5, 2026

Abstract

Empirical studies of architectural change read a project's own records and assume the record traces the work. We measure that assumption in three record channels across two ecosystems and find it fails in every one.

We probed 38 Apache projects against a traceability bar of 0.80, pre-registered before any project was cloned and never moved. **Twelve passed, and no project outside the Hadoop ecosystem cleared the bar** — the best reaching 74.8%, against a median of 63.6% across all 38 and 44.8% across the 26 rejected. The ecosystem label is a hand classification; we tested two measured generational variables in its place and neither separates the corpus as well, so the label is reported as a judgment rather than a finding. The channel the literature publishes is not the channel a ticket-anchored study needs. The **commit-side rate** — what fraction of commits cite a ticket — runs 82.6–98.3% across the eligible twelve, while the **ticket realisation rate** — what fraction of tickets ever receive a citing commit — runs 12.0–69.0%. Apache Hive cites a ticket in 97.0% of its 18,213 commits and realises 55.8% of its 29,635 tickets.

Most of that gap is arithmetic, not discipline. A citing commit adds at most one new distinct ticket, so the ticket-side rate cannot exceed `commit-side × commits / tickets`. That ceiling binds for all twelve eligible projects, ranging 13.7–81.6%, and every project reaches 80–95% of it: the 5.8× spread in the ticket-side rate is a 6.0× spread in the ceiling and only a 1.19× spread in what is left. A tracker accumulates tickets faster than a repository accumulates commits, and below one commit per ticket the two rates are not commensurable.

Extending the computation to all 38 projects, including the 26 the bar rejected, the two rates show **no detectable association** — Spearman $\rho = -0.010$ over the 33 with a usable denominator, 95% CI $[-0.35, +0.34]$, and the study can detect $|\rho| \geq 0.47$ at 80% power. **A strong positive relationship is ruled out; a null is not established.** Median realisation is 52.6% among projects the bar accepted and 53.2% among those it rejected, and the highest rate in the probe (84.8%) belongs to a project rejected at 36.0% commit-side. On the twelve projects measured exactly and live the same correlation is +0.518, and we report the disagreement rather than resolving it.

The same invisibility appears in a second ecosystem and two further channels: in a TypeScript corpus, 3 of 96 architectural commits mention refactoring, 7% link an issue, and 69% never pass through code review — and the 31% that do are five times larger and four times more abstraction-heavy than those that do not, so the visible minority is not a random sample of the whole.

We give a taxonomy of six mechanisms by which a project silently fails to support an issue-linked study. Three are disqualifying — the tracker is displaced by GitHub Issues, no source repository exists, the tracker is downstream of the repository. Three are **silent**: a convention that changed mid-history, a monorepo needing multi-key matching, and a cited

key with no project record in the tracker each yield a plausible low number rather than an error. A single-key probe reads Hadoop at 26.2% where a seven-key probe of the same commits reads 97.8%, and nothing in the first result signals the key set is wrong.

None of the six is expressible in any published sampling frame. GHS indexed 735,669 repositories when it was published in 2021, and a record carries 35 fields today, of which only two touch issues and both are GitHub-issue counts. Project eligibility for issue-linked research cannot be established from metadata; it requires reading commit messages from the project itself, per project, before any outcome is measured.

We make no first-to-measure claim: per-project linkage rates are published by SEOSS 33 for 33 projects and by Rath et al. (ICSE'18) for six. What is new is a pre-registered numeric bar with reported attrition and named rejections, both reference channels counted together, the ticket realisation rate named and measured across a whole probe rather than its survivors, and the six-mode taxonomy.

What to read first. This paper carries three tables and one figure. They are the paper; the prose is the argument around them.

- **Table 1** — corpus eligibility across the 38 probed projects, both traceability channels, with commits per ticket, the arithmetic ceiling of Section 3.1.4 and the share of it reached.
- **Table 2** — three-channel visibility of architectural change across two ecosystems. Every cell is a different quantity with its own denominator and no statistic is computed across them.
- **Table 3** — the ticket realisation rate across all 38 probed projects, with the note column recording every reason a row should not be read at face value.
- **Figure 1** — 38 to 12 to one ecosystem, plus the six hypotheses that did not hold.

---

1 Introduction

Empirical work on architectural change reads a project's own records — commit messages, issue trackers, pull requests — and assumes the record is a reasonably faithful trace of the work. Studies of refactoring motivation, of technical-debt repayment, of architectural erosion and of review effort are all built on that assumption, and it is rarely stated, still more rarely measured. This paper measures it, in three record channels across two ecosystems, and finds it false in every channel tested.

How we arrived at a methods paper. The study began as an attempt to measure signal-to-action latency: what, if anything, in a project's records precedes the decision to restructure code. Six operationalisations were tested and closed, each with a named cause — a Cox model on review discussion, retracted as a discussion-volume confound; module centrality as a predictor of hesitation, null once the module tier is controlled; maintainer concentration, collinear with module size at $\rho = -0.86$; an abstraction-versus-relocation split, $\times 1.54$ $[0.97, 2.43]$ fully adjusted; a decade-long process-improvement trend that holds to 2019 and dissolves after 2020; and a portable module-tier rule that replicated 0 of 3 in a held-out corpus. The details are in the replication package and are not results of this paper. What survived is the observation that made all six fragile: **the sampling frame itself is the finding.** We could not assemble a corpus that could carry any of those tests, and the reasons why turned out to be measurable, general, and absent from every published sampling tool.

The core measurement. Traceability between issues and commits is published one way and needed the other way. The **commit-side rate** — what fraction of a project's commits cite a ticket — is what the literature reports and what selection criteria use. The **ticket realisation rate** — what fraction of a project's tickets ever receive a citing commit — is what a ticket-anchored study actually depends on, because such a study samples tickets and then looks for the work. The two diverge sharply and in a direction that flatters the corpus. Apache Hive cites a ticket in **97.0%** of its 18,213 commits — nearly perfect by any selection criterion — and realises **55.8%** of its 29,635 tickets.

And most of that gap turns out to be arithmetic. A commit that cites a ticket adds at most one *new* distinct ticket, so the ticket-side rate is capped at `commit-side × commits / tickets`. That ceiling binds for every eligible project, running 13.7–81.6%, and every one of them reaches 80–95% of it. What reads as a discipline gap is mostly a tracker accumulating tickets faster than a repository accumulates commits — which is a fact about the two artifacts, not about the people using them, and which no published linkage rate exposes.

Contributions. Stated against what is already published rather than against an assumed gap. We make **no first-to-measure claim**: Rath & Mäder's SEOSS 33 publishes per-project linkage as an explicit table column for 33 projects, and Rath et al. (ICSE'18) publishes both directions for six. An earlier version of this work claimed novelty on that ground; the claim was checked, found false, and withdrawn (§2.1).

1. **A pre-registered numeric eligibility bar, applied before any outcome, with reported attrition and the rejected candidates named.** Prior work selects on trace links informally — "a project shall continuously capture trace links", "each largely followed the practice of tagging commits with issue IDs" — and reports rates afterwards. Neither states a threshold, an attrition count, or which candidates were rejected. Ours is fixed at 0.80, committed before any project was cloned, and never moved.
2. **A population finding: 12 of 38 pass, and no project outside one ecosystem clears the bar.** The best outside it, James, reaches 74.8%, against a median of 63.6% across all 38 and 44.8% across the 26 rejected. The ecosystem label is a hand classification

and we say so: two measured generational variables were tested in its place and neither separates the corpus as well (§4.1). SEOSS's own table contains the ingredients — Apache projects at the top, JBoss at the bottom — but the inference is not drawn there.

3. **Both reference channels measured together.** Counting GitHub-issue references alongside Jira keys is what shows that the bar selects a *tracker* rather than a discipline: seven of the dropped projects cite GitHub issues in more than half of their commits.
4. **The arithmetic ceiling that bounds the ticket-side rate**, and the decomposition of that rate into a ceiling set by commits-per-ticket and a residual that measures citation discipline. In this corpus the residual is nearly constant, which is what the divergence actually consists of.
5. **The ticket-side rate measured across the whole probe rather than only the survivors** — including the 26 projects that failed the commit-side bar. A strong positive association between the two rates is ruled out; a null is not established, and the live and frozen measurements disagree in sign (`paper/DIRECTION_TENSION.md`).
6. **A six-mode taxonomy of the ways a project silently fails to support an issue-linked study**, three of them disqualifying and three of them producing a plausible wrong number rather than an error.
7. **A second ecosystem and two further channels**, which is what makes the result more than an Apache artifact: in a TypeScript corpus, 3 of 96 architectural commits mention refactoring, 7% link an issue, and 69% never pass through code review at all.

What this paper is not. It is not a study of refactoring friction; the exploratory Hadoop results live in the replication package and none of them is claimed here. It does not propose the successor design the corpus limits point towards. And it does not argue that the projects it drops are badly run — a project that moved to GitHub Issues is not failing at anything except a convention this kind of research happens to need.

2 Related work

Drafted from `paper/PRIOR_WORK.md` (`215b10f`) and `paper/SATD_NOVELTY.md` (`5b50ef1`). Reorganisation of verified findings, not new argument.

2.1 Per-project linkage rates are already published

This paper makes no first-to-measure claim, and the claim it originally made was withdrawn. Two prior works publish per-project issue–commit linkage rates, one of them as an explicit table column.

Rath & Mäder 2019, SEOSS 33 (*Data in Brief* 25:104005) publishes, per project, change-set count and **"Linked Change Sets [%]"** for 33 projects — the same quantity as our commit-side rate. Their spread is 8.11%–97.13%; ours is 0.01%–98.3% across 38. Four of five overlapping projects agree within **3.3pp** on corpora seven years apart:

project	SEOSS 2019	this probe 2026	Δ
Hadoop	97.13% (27,776 commits)	97.8% (28,290, 7-key)	+0.7pp
Hive	96.34% (11,179)	97.0% (18,213)	+0.7pp
HBase	90.06% (14,331)	92.5% (21,220)	+2.5pp
ZooKeeper	87.12% (1,600)	90.4% (2,718)	+3.3pp
Flink	41.98% (12,419)	66.0% (38,219)	**+24.1pp**

That agreement is **independent cross-corpus validation of the measure** and is treated here as such. Flink's 24.1pp gap is scope rather than error, and unlike in earlier drafts this is now tested rather than asserted: restricting our probe to the **earliest 12,419 commits** — the change-set count SEOSS publishes, and the only quantity the two studies share — gives **5,214 / 12,419 = 41.9841%** against their **41.98%**, a gap of **+0.0041pp**. All five overlapping projects agree once scope is matched.

On the exactness of that match, which invites suspicion and should not. The quantity is a deterministic count — commits whose message matches a key pattern, over a fixed and identically bounded commit range — not an estimate, and it carries no sampling error. Two correct implementations of the same well-specified count *should* agree exactly; a disagreement would indicate a difference in specification, not noise. Exact agreement is only suspicious between estimators that have sampling error, and this has none. The two figures also reach us by independent paths: SEOSS's 41.98% is transcribed from their published table, and 41.9841% is a fresh scan of a clone at a pinned sha. **What the match confirms is scope alignment, and nothing beyond it.**

SEOSS's selection criteria matter for our argument: a project must "continuously capture vertical and horizontal trace links among these artifacts". So a traceability criterion **is** used for selection — but as a qualitative requirement. No cut-off is stated, no rejected candidates are reported, and having selected on trace links the dataset still admits Errai at 8.11%.

Rath et al., ICSE 2018 (*Traceability in the Wild*, arXiv:1804.02433) publishes **both directions** for six Git+Jira projects, and is the closest precedent to our divergence result. Commit side: "approximately 48% of the commits were not linked to any issue", with a per-project spread from 15% unlinked in Derby to ~76% in Maven. Ticket side: "approximately 43.3% of improvements and 42.4% of bugs have no commits associated with them" — Derby works out to 51.8% ticket-side coverage. Our 12 passing projects run 12.0%–69.0% ticket-side, straddling their figure. Their sentence — "different practices exist across different projects, leading to huge disparities in the extent to which issue tags are added to commit messages" — is our corpus-feasibility finding stated qualitatively in 2018.

Selection there was also informal: the six were chosen because each "largely followed the practice of tagging commits with issue IDs". Again no threshold.

Vieira et al. 2019 (PROMISE'19, 55 Apache projects, >70,000 bug reports) may or may not report per-project linkage. **Unverified:** ACM DL, ResearchGate and figshare all returned 403 to unauthenticated fetches, so neither the paper body nor the package manifest could be read. Recorded as unverified rather than characterised. Note its selection is already conditioned on *resolution = Fixed*, which pre-selects tickets that were worked, so any rate it reports would not be comparable to ours without care.

2.2 What is therefore new here

Positioned against the above rather than against an assumed gap:

1. **A pre-registered numeric bar applied before any outcome, with reported attrition.** Both prior works select on trace links informally and report the rate afterwards; neither states a threshold, an attrition count, or which candidates were rejected. Ours is fixed in `predictions/PREDICTIONS.md` (`ca076a9`) before any project was cloned, and never moved.

2. **The population finding.** 12 of 38 pass and no project outside one ecosystem clears the bar. SEOSS's own table contains the ingredients — Apache at the top, JBoss at the

bottom — but does not draw the inference. The ecosystem label is a hand classification and §4.1 reports the measured variables that failed to replace it.

3. **Both reference channels measured together.** No prior work counts GitHub-issue references alongside Jira keys, which is what shows the bar selects a *tracker* rather than a discipline.

4. **The arithmetic ceiling.** The bound TRR $\leq$ CSR $\times$ commits / tickets, the decomposition of the ticket-side rate into that ceiling and a residual, and the finding that in this corpus the residual is nearly constant while the ceiling varies six-fold (§3.1.4, §4.2). No prior work reports commits per ticket alongside a linkage rate, which is what makes the two rates look commensurable when they are not.

5. **The ticket-side rate measured across the whole probe**, including the 26 projects the bar **rejected** — Rath 2018 reports it per issue *type*, and only for the six projects it selected. *(Whether the quantity is also newly **named** depends on the Bachmann determination in §2.4 and is **[PENDING]**.)*

6. **The six-mode taxonomy**, and that none of it is expressible in any published frame.

2.3 The standard sampling frame cannot express the criterion

Dabic et al. 2021, GHS (*Sampling Projects in GitHub for MSR Studies*, MSR'21) is the standard sampling tool and indexes **735,669 repositories**. A record carries **35 fields**, queried from the live API rather than read off the paper's Table I, which is an image. Only `totalIssues` and `openIssues` touch issues at all, and both are GitHub-issue counts. There is no field for issue-tracker type, external tracker usage, issue–commit linkage, traceability, or commit-message convention. The one adjacent feature, filtering by issue label, is described in the paper as "still under development".

The consequence is the actionable one: a researcher sampling with GHS gets stars, commits, contributors and license, then discovers post hoc that 26 of 38 candidates are unusable. That is what happened here.

2.4 The bug-side linkage literature, and what it already settled

The commit-side/ticket-side asymmetry this paper measures was studied on the bug-side a decade and a half ago, and that literature is the direct ancestor of §4.2. It is set out here rather than merely listed, because two of its findings bound what this paper can claim.

Bachmann et al. (FSE'10), *The Missing Links: Bugs and Bug-fix Commits* — Adrian Bachmann, Christian Bird, Foyzur Rahman, Premkumar Devanbu and Abraham Bernstein — is the closest ancestor. They engaged a core Apache HTTP Server developer to annotate **493 commits over a six-week period** exhaustively, using a purpose-built tool (Linkster), to establish ground truth rather than infer it from commit messages. Against that ground truth they found that **only 47.6% of bug-fix-related commits are documented in the bug tracking database**. Their target is the completeness of the *link*, established by expert annotation on one project over one window; ours is a per-project rate over a whole tracker, established mechanically. Their design is far stronger on ground truth and far narrower in scope; ours is the reverse.

> [**DETERMINATION PENDING**] Whether the ticket realisation rate of §3.1 is the same quantity Bachmann et al. measured, or a distinct one, is **not settled in this draft**. The argument for distinctness is that §3.1.1 admits every issue type and every status and conditions on nothing, whereas Bachmann conditions on bugs and bug-fix commits. That argument has not been adjudicated against the paper's own

text, and the naming claim in §3.1 stands or falls with it. See `paper/REVISION_LOG.md`, GATE.

Bird et al. (ESEC/FSE'09), *Fair and Balanced? Bias in Bug-Fix Datasets* — C. Bird, A. Bachmann, E. Aune, J. Duffy, A. Bernstein, V. Filkov and P. Devanbu — is the reason any of this matters. Missing links are not missing at random, so a dataset built from linked records is a biased sample of the work, and models fitted to it inherit the bias. **This is the same argument our §7.3 makes for architectural change**, arrived at independently and seventeen years later, and we cite it as the prior statement of the principle rather than as a parallel.

Nguyen, Adams and Hassan (WCRE'10), *A Case Study of Bias in Bug-Fix Datasets* replicates that bias analysis. *(The review that prompted this revision cited this work as MSR'10 and characterised it as a replication on a system with near-perfect linkage; the venue is WCRE'10, and the near-perfect-linkage characterisation could not be verified from an accessible copy, so it is not asserted here — see `paper/REVISION_LOG.md`.)*

Herzig, Just and Zeller (ICSE'13), *It's not a bug, it's a feature: how misclassification impacts bug prediction* bears directly on §3.1.1's decision to admit all issue types. In a manual examination of **more than 7,000 issue reports across five open-source projects they found 33.8% misclassified** — filed as bugs but resolving to a feature, a documentation update or an internal refactoring — and **39% of files marked defective never had a bug**. Our denominator admits every type precisely so that no misclassification can move a ticket in or out of it. That immunises the ticket realisation rate against the Herzig effect, and it is also why our rate is *not* comparable to any rate computed on a resolution-filtered or type-filtered population, including Vieira et al.'s.

Automated link recovery is the standing partial answer to modes 4–6. ReLink (Wu, Zhang, Kim and Cheung, ESEC/FSE'11) learns the features of explicit links — time proximity, author identity, textual similarity between the bug report and the change — and recovers missing ones at accuracy well above the traditional regex heuristics, and a substantial literature has followed it. §5.5 and §7.2 argue that a project's key set cannot be recovered from its tracker, which remains true: ReLink and its successors recover *links*, not *key sets*, and they operate on a project already known to be a candidate. But the unqualified claim that missing links are unrecoverable would overstate the gap, and §5.5 is qualified accordingly.

2.5 Refactoring and self-admitted technical debt

Included because it bounds what this paper claims. **Iammarino et al. (2021, JSS)** and **Esfandiari & Sami (ICCKE 2023)** both study SATD and refactoring strictly at same-commit co-occurrence — Iammarino over four projects with no temporal analysis, its tightest cut being same-file at $n = 201$; Esfandiari over 77 projects, already reporting *move class* as the refactoring most associated with debt activity. Neither measures an interval. An architectural-debt time-to-fix literature is separately active (arXiv:2605.16133, arXiv:2501.15387).

Two things follow. The record channels this paper measures are the channels that literature depends on, so its exposure is ours. And the successor design this study points to is **not proposed here**; it is a separate registered report gated on external judgment (`paper/SATD_NOVELTY.md`).

2.6 Detector validity

RefactoringMiner is the detector. Its TypeScript support was complete 2026-05-24, two months old at measurement, and has no independent validation we could find in the literature. One defect was traced and reported upstream: 160 `interface` → `class` false positives in a single commit, from

type aliases having no representation in the tool's class model (tsantalis/RefactoringMiner#1124, filed 2026-07-25). That finding is a separate paper and appears here only as a bound on the TypeScript column of Table 2.

3 Method

3.1 Two traceability rates, and only one of them is published

The literature reports one number and issue-anchored studies depend on another. Both are stated here as definitions so that the divergence in §4 is a comparison between named quantities rather than between two informal readings of the word "traceability".

Let p be a project with a set of Jira project keys K_p, a git repository observed at a pinned commit H_p, and a tracker read at a snapshot time $\mathbf{T}$. Write `cites(c)` for the set of issue keys appearing in commit c's message.

Definition 1 — commit-side rate. The fraction of commits reachable from H_p whose message cites at least one key with a prefix in K_p:

$$\mathrm{CSR}(p) = \frac{\left|\{c : c \longrightarrow H_p, \mathrm{cites}(c) \cap K_p \neq \emptyset\}\right|}{\left|\{c : c \longrightarrow H_p\}\right|}$$

This is the quantity Rath & Mäder publish per project as *"Linked Change Sets [%]"* (SEOSS 33) and the quantity Rath et al. (ICSE'18) report as "approximately 48% of the commits were not linked to any issue". It is what the corpus-selection bar in this study is defined on, and it is not novel here.

Definition 2 — ticket realisation rate (TRR). The fraction of the project's tracked issues that are ever cited by at least one commit:

$$\mathrm{TRR}(p) = \frac{\left|\{k \in \mathrm{Tickets}(p,T) : \exists c \longrightarrow H_p, k \in \mathrm{cites}(c)\}\right|}{\left|\mathrm{Tickets}(p,T)\right|}$$

The name is introduced here because the quantity has been reported without one. Rath et al. (ICSE'18) measure it — "approximately 43.3% of improvements and 42.4% of bugs have no commits associated with them" — as a property of an issue type rather than as a project-level rate with a name, and the divergence between it and CSR is not framed anywhere as a constraint on corpus selection.

"Realisation" is a claim about the record, not about the work. TRR counts a ticket as realised when the repository's own record points back to it. A ticket can be resolved, implemented and shipped without any commit naming it, and such a ticket is counted as unrealised. That is the intended reading: the measure is of what a researcher can *recover*, not of what a project *did*.

3.1.1 What counts as a ticket

Denominator membership is deliberately permissive, because a ticket-anchored study samples from the whole tracker before it knows which tickets are useful:

- **All issue types.** Bug, Improvement, New Feature, Task, Test, Wish and Sub-task alike. **Sub-tasks are included**, because they carry their own key and are cited in commit messages in their own right; excluding them would drop the keys most likely to be cited and inflate the rate.

- **All statuses and resolutions.** Nothing is conditioned on `resolution = Fixed`. This matters for comparability: Vieira et al. (PROMISE'19) select on resolution before measuring, which pre-selects tickets that were worked, so any rate computed that way is not comparable to TRR without adjustment.
- **Keys, not names.** Membership is by the issue's key prefix. Measured across the 2,686,282-issue public Jira corpus, 326 project ids carry more than one distinct name while 0 carry more than one key, so a name-based denominator is not stable and a key-based one is (`paper/numbers.md` §5).
- **Issues that left the project are not in it.** An issue moved out of p before T no longer has a p-key in the tracker and is not counted; one moved in is counted under its current key. This is a property of the snapshot, not a choice.

3.1.2 What counts as "ever received a commit"

A ticket is realised if **at least one** commit reachable from H_p carries a token matching `\b(?:K1|K2|...)-\d+\b` equal to its key, in the commit **subject or body**. Nothing else is read: not the diff, not the branch name, not a pull request body, not the tracker's own link fields.

The matcher's precision was measured rather than assumed: **195 of 200 = 97.5%** (Wilson 95% CI 94.3–98.9%) on a seeded 200-commit manual sample across the 12 eligible projects, corpus-weighted 97.7% (`paper/matcher_validation.md`). Two of the five enumerated failure categories could not have fired — `version_string` is near-unreachable given a pattern demanding the upper-case key followed by `-` and digits, and `foreign_key` is unreachable for 11 of the 12, because each project is probed with its own key set and only Ozone is multi-key with both keys its own. **The single-key → multi-key result on Hadoop is therefore not validated by that sample**, Hadoop being the only true monorepo in the study and not among the 12.

One construct limit is recorded rather than corrected: **a key matched from a branch name counts as a citation.** Merge subjects (`Merge branch 'master' into KNOX-998-Package_Restructuring`) and svn trailers (`.../branches/TEZ-1@1471779`) both occur in the sample. The ticket association is correct; the commit need not implement the ticket. The five failure categories were fixed before labelling and do not cover this case, so it was counted genuine and is disclosed here instead of being assigned an invented category.

3.1.3 How the denominator is bounded in time

This is the definitional detail that decides whether TRR is interpretable, and it is the one most easily got wrong.

TRR compares a tracker read at T against a repository read at H_p. **The measure requires T $\leq$ date(H_p)**, so that every ticket in the denominator has had the whole interval from its creation to date(H_p) in which to receive a commit. If T runs ahead of the repository, the last tickets filed are structurally incapable of being realised and TRR is biased downward by an amount that depends on the project's filing rate — an artefact that looks exactly like poor traceability.

Two instantiations are used, and both satisfy the constraint:

	tracker snapshot T	**repository H_p**	**coverage**
TRR_{live}	Apache Jira read 2026-07-25, issue **keys only**	pinned shas of 2026-07-25	the 12 eligible projects
$\text{TRR}_{\text{frozen}}$	the Public Jira Dataset (Zenodo 15719919), a snapshot predating every pinned sha	the same pinned shas	all 38 probed projects

For TRR_{live}, T $\approx$ date(H_p): the two were read the same day, so the newest tickets are the censored ones and TRR_{live} is a slight underestimate. The restriction to issue keys is enforced mechanically, not by care — the field list sent to the API is asserted to be `["key"]`, every returned issue is screened against a forbidden-field set and a hit aborts the run rather than being filtered out, and only the key string survives the parse loop. That is what makes the measurement admissible over a held-out corpus: key existence is not an outcome and nothing in it can be turned into a duration.

For $\text{TRR}_{\text{frozen}}$, T is years behind H_p, which is the safe direction but introduces a different problem: the snapshot's date is not recoverable per project from the parsed artifact. Membership is therefore defined in **issue-number space** rather than in date space. Jira numbers issues sequentially from 1 within a project, so a tracker holding N_p issues at T holds approximately the first N_p numbers, and the denominator is N_p while the numerator counts only cited keys numbered $\leq N_p$. The two definitions coincide except for issues moved between projects, which perturb the correspondence between count and highest number. §4.3 measures the size of that perturbation rather than assuming it away.

3.1.4 The arithmetic ceiling, and what is left once it is removed

CSR and TRR are not independent quantities, and the constraint between them is exact rather than statistical. It is stated here because §4.2 and §4.3 are organised around it.

A commit either cites no key of p, in which case it contributes nothing to the numerator of TRR, or cites at least one — and however many it cites, it can add **at most one ticket that no earlier commit had already cited**, in the limiting case where every citing commit introduces a fresh key. So the number of distinct realised tickets is bounded by the number of citing commits:

{ k ∈ Tickets(p, T) : k realised }	**≤ CSR(p)** ·	C_p

and dividing by |Tickets(p, T)|:

$$\text{TRR}(p) \leq \frac{\text{CSR}(p) \cdot |C_p|}{|\text{Tickets}(p, T)|} \equiv \text{ceiling}(p)$$

The bound is tight — equality holds when citing commits map one-to-one onto distinct previously-uncited tickets — and it is reached in practice only if no ticket ever receives two commits.

Two consequences, and the second is why this matters.

First, **`commits / tickets` is a scale factor the two rates do not share.** A project with 0.16 commits per ticket cannot exceed a 16% ticket-side rate at *any* commit-side rate, including 100%. Comparing CSR and TRR without it compares a ratio to a ratio with a different denominator.

Second, it decomposes TRR into a part that is arithmetic and a part that is behaviour:

$$\text{fill}(p) = \frac{\text{TRR}(p)}{\text{ceiling}(p)} \in [0, 1]$$

`fill` is the share of the attainable maximum actually reached — how efficiently a project's citing commits spread across distinct tickets rather than piling onto a few. **`fill` is the quantity a claim about citation discipline needs**; `ceiling` is a property of how much code the project

writes per ticket it files. Tables 1 and 3 report both. §4.2 shows that in this corpus almost all of the ticket-side variation is ceiling and almost none of it is fill.

3.2 Corpus construction

38 Apache candidates, each Maven-built, Jira-tracked and multi-module. Each was cloned `--filter=blob:none --no-checkout` — the probe needs `git log` and nothing else, so a working tree is pure cost — and probed with `scripts/traceability_probe.py`. Per-project HEAD shas are pinned in `paper/traceability_probe.json`, so any re-run is exactly diffable against this one.

Hadoop itself is not one of the 38. It is the corpus the exploratory work was done on, and it is used here only as the worked example for multi-key matching. Every rate in Tables 1 and 3 is from a project Hadoop is not.

Key prefixes were detected empirically from commit messages, not assumed. This is not a refinement; it decides the answer. A single-key probe reads Hadoop at **26.2%**, against **92.3%** on a four-key probe and **97.8%** on a seven-key probe of the same 28,290 commits (§5, modes 5 and 6). The three figures are the same measurement under three key sets, not a rate and a correction to it; 97.8% is the most complete of them, and the paper quotes whichever key set it names. Evergreen reads 71.7% under a single-key probe for the same reason. Seven of the 38 needed a second key (`CALCITE,OPTIQ`, `HDDS,OZONE`, `KARAF,FELIX`, `PINOT,THIRDEYE`, `ROCKETMQ,RIP`, `TOMEE,OPENEJB`, `JAMES,MAILBOX`).

One of those second keys was never used. `OZONE` appears in Ozone's probed key set and is cited by **zero** commits; every Jira reference in that repository is an `HDDS` key. It is retained in the probe because the key set was fixed from a prefix scan before the counts were read, and removing it afterwards would be selection on the outcome. §5.3 counts it among the six probed keys with no project record in the frozen tracker corpus, which is a different fact about the same key: it is neither cited in git nor present in the tracker.

Both reference channels are counted. GitHub-issue references (`#NNN`, `GH-NNN`) are counted per repository alongside Jira keys. This is what turns each drop reason from an inference into a measurement: `github_issue_references_dominate` means the GitHub count exceeds the Jira count in that repository, and nine projects qualify.

3.3 The bar, and the discipline around it

CSR $\geq$ 0.80, fixed in `predictions/PREDICTIONS.md` (`ca076a9`) **before any project was cloned, and never moved.** Lowering it to 60% would have admitted nine more projects and widened the corpus beyond a single ecosystem. It was not lowered.

The same discipline was applied where it hurt more. A separate frozen threshold — vendor share $\geq$ 0.40 for the module-tier rule — was held through a held-out replication in which lowering it would have rescued two of five projects (HBase, maximum share 0.33; Phoenix, which flagged nothing). It was not lowered, and the rule then failed 0 of 3 where it could be tested (`replication/REPLICATION.md`). Both facts are stated because a pre-registered threshold that is never tested against temptation is not evidence of anything.

Held-out discipline is mechanical. Outcome data was never observed for seven projects — Ozone, Tez, ZooKeeper, Ranger, Oozie, Knox, Sqoop. The rule is written down (`PROJECT_STATE.md` §3); the ticket-side script requests one Jira field and aborts if it receives another; the matcher-validation script asks git for `%H`, `%s` and `%b` and nothing else, so it *cannot* compute a duration. HBase and Phoenix were dropped from the held-out corpus on 2026-07-30 and

quarantined, because their committed artifacts are one subtraction away from per-ticket latency even though no outcome was ever observed for either; their coverage numbers remain usable and are reported.

3.4 The detector, and the second ecosystem

Architectural episodes in the Apache corpus were detected with **RefactoringMiner 3.1.4** over 8,919 Hadoop commits, at AST level, never from message text. The second ecosystem — `BearStudio/start-ui-web`, TypeScript, 1,199 commits — is **exploratory throughout**, and only counts and proportions from it enter this paper, never a test. Its detector caveats are load-bearing and are given in full in §6.7: TypeScript support was two months old at measurement, has no independent validation in the literature, and `Change Type Declaration Kind` was excluded as a false positive (160 instances in one commit, upstream issue #1124).

4 Results

4.1 Corpus eligibility is the binding constraint

Twelve of 38 Apache candidates clear the pre-registered 0.80 commit-side bar, and every one of them is Hadoop-ecosystem under the classification set out below (Table 1; `figures/eligibility_funnel.png` panel A). The bar was fixed before any project was cloned and never moved.

No project outside the Hadoop ecosystem clears it. The best of them, James, reaches **74.8%**. The spread runs from ShardingSphere at 0.01% to Ozone at 98.3%, with a median of **63.6%** across all 38 and **44.8%** across the 26 the bar rejected.

The ecosystem label is a hand classification, and we tested whether a measured variable does the same work. It does not. "Hadoop-ecosystem" is not a field in any dataset; one of the authors assigned it. The natural replacement is project generation — free metadata, no judgment, and a mechanism, since projects predating the 2019–20 migration of ASF development to GitHub pull requests had longer under the Jira-citation convention. Two generational variables were tried (`scripts/era_separation.py`):

variable	coverage	separation
Hadoop-ecosystem label (hand)	38 of 38	accuracy **0.868**, recall 1.000, precision 0.706, Fisher $p = 2.3 \times 10^{-6}$
repository start year (computed from the clones)	38 of 38	AUC **0.611**, $p = 0.279$, best-threshold accuracy 0.711
Apache Incubator graduation date	**24 of 38**	AUC 0.289, $p = 0.098$, best-threshold accuracy 0.625

Neither generational variable separates the corpus as well as the hand label, and the graduation date is **missing precisely where it would be needed**: four of the twelve passing projects — Hive, HBase, ZooKeeper and Ozone — entered the ASF as Hadoop subprojects rather than as incubator podlings and have no graduation date at all. The free metadata is not free for the group that matters.

We therefore keep the hand label and mark it as a judgment. The claim that survives without it is the measured one: 12 of 38 pass, and the twelve are concentrated in a way no available metadata field predicts. Reclassifying **Drill** — which requires neither HDFS nor YARN and is arguably outside the ecosystem — would falsify the "all twelve" form of the claim and put

a non-ecosystem project above the bar. Kylin and Sqoop are the same shape of exposure; Atlas is not, because at 78.1% it fails the bar under either classification.

Ecosystem adjacency is not sufficient either — **Parquet fails at 29.1% and Accumulo at 43.0%**, both squarely inside the Hadoop dependency graph. What the bar selects is a specific commit-hygiene convention, not a dependency relationship and not a quality level.

The drop reasons are measured rather than inferred, because both reference channels are counted per repository. Nine projects are dropped with `github_issue_references_dominate`, meaning the GitHub-issue count exceeds the Jira-key count in that repository — ShardingSphere carries 30,746 GitHub-issue references against 5 Jira keys in 49,111 commits. Four fall in `below_bar_narrowly` ($\geq$70%) and the remainder in `low_commit_message_hygiene`. The industry-donated projects are the sharpest case: ShardingSphere 0.01%, SkyWalking 0.3%, Pinot 5.2%, RocketMQ 5.7%, Dubbo 11.0%. They never adopted the Jira-citation convention, and the attrition is therefore concentrated in the newest projects — which is the direction that matters for anyone building a corpus now.

4.2 The channel that is published is not the channel that is needed — and the gap is mostly arithmetic

Across the same twelve projects, the **commit-side rate runs 82.6–98.3%** while the **ticket realisation rate runs 12.0–69.0%, with none above 69.0%** (Definition 1 and Definition 2, §3.1; Table 1). The headline case:

> **Apache Hive cites a ticket in 97.0% of its 18,213 commits, and 55.8% of its 29,635 tickets are ever cited by one.** Nearly perfect commit-side hygiene, and still nearly half the tracker is invisible to a ticket-anchored design.

Most of that gap is not a discipline gap. It is the ceiling (§ 3.1.4). Hive has **0.61 commits per ticket**, so at 97.0% commit-side its ticket-side rate cannot exceed **59.6%** whatever anyone does — and it reaches **0.94** of that. The pattern holds across all twelve:

- the **ceiling binds for every one of them**, running 13.7% (Kylin) to 81.6% (Ranger) — a 6.0× spread;
- the **fill is flat**: median **0.89**, range **0.80** (Ranger) to **0.95** (Drill), a spread of only 1.19×;
- so the 5.8× spread in the ticket-side rate is 6.0× **ceiling and** 1.19× **fill**.

Ranked by fill rather than by rate, the ordering changes almost completely: Drill (43.2%, fill 0.95) sits above Ranger (65.4%, fill 0.80). *Every project in the corpus is reaching most of what its commit supply permits.*

This is a mechanism, and it replaces the reading the divergence invites. The tempting story — "these projects file tickets they never work on" — is not what the numbers show. What they show is that **a tracker accumulates tickets faster than a repository accumulates commits**, and once fewer than one commit exists per ticket the ticket-side rate is capped below the commit-side rate by construction. The two rates were never commensurable, and reporting one as though it licensed the other is the error, not the projects' behaviour.

The consequence for design is unchanged and now has a reason. A study that samples commits and looks up their tickets can work in any of these twelve. A study that samples tickets and looks for the work is bounded by `commit-side × commits / tickets` before it begins, and that quantity is not reported anywhere in the literature — including by us, until now.

4.2.1 Kylin: the sharpest number in the corpus, and why it is withdrawn as the example

Earlier drafts led with Kylin — 83.9% commit-side against 12.0% ticket-side, "seven of every eight tickets with no commit at all". **That example is withdrawn.** The pinned commit reaches **968 commits beginning 2022-08-01**, which is **7.5% of the 12,937 commits on the repository's refs**, and the repository itself begins 2014-05-13. The `apache/kylin` default branch was re-initialised for Kylin 5; the earlier history is on other branches.

So Kylin's 12.0% is measured over a **four-year commit window against a twelve-year, 5,931-issue tracker**, and its 0.16 commits per ticket — the lowest in the corpus by a factor of two — is a fact about the branch, not about Kylin's traceability. Its fill is **0.87**, squarely at the corpus median: *by the measure that isolates discipline, Kylin is unremarkable.*

Three other projects have a pinned branch that starts after their repository does — Jena (49.0% of refs), Karaf (45.4%) and James (89.4%, by eight days) — and none is near Kylin's severity. The clone-depth column is in `paper/revision_metrics.json`; the check is `scripts/revision_metrics.py`.

Kylin is retained in every table, because the bar was applied to it before any of this was known and removing it now would be selection on the outcome. It is flagged wherever it appears, and no claim in this paper rests on it.

4.3 The divergence is a property of the population, not of the survivors

Reporting §4.2 on the twelve that passed leaves the obvious objection open: the range 12.0–69.0% is *within-passing variation*, and says nothing about the 26 projects below the bar, whose ticket realisation rates were never measured. That objection is answered here by measuring all 38. **Table 3 (`paper/table3_ticket_side.md`) is the result**: one row per probed project, with the commit-side rate, commits per ticket, the §3.1.4 ceiling, the ticket realisation rate, and the fill, plus a note column recording every reason a row should not be read at face value.

Method. The ticket realisation rate needs a tracker snapshot. Re-reading the live tracker for 26 more projects would have dated those denominators weeks after the twelve already measured, so the extended computation instead takes its denominators from the frozen public Jira corpus and aligns numerator and denominator in issue-number space (§3.1.3). **The estimator is validated before it is used**: run against the live denominators for the twelve projects where the exact rate is already known, it reproduces that rate to a **mean absolute error of 0.45pp** across all twelve, with a **worst case of 2.14pp** (Kylin, where the tracker's issue numbering has enough gaps that number-capping undercounts). All 38 projects were then measured at their pinned shas.

The estimator, and the honest version of its validation. Two approximations are stacked here: *number-capping* (using $|\{\text{cited keys} \leq N\}| \,/\, N$ rather than intersecting with the real key set) and *snapshot substitution* (using the frozen tracker's N rather than a live one). Earlier drafts reported **0.45pp mean / 2.14pp max**, which validates only the first. **Table 3 uses both**, and the end-to-end error against the twelve published exact rates is:

	mean	**max**	**within 3pp**
number-capping alone	0.45pp	2.14pp	12 of 12
number-capping + snapshot substitution (what Table 3 uses)	**1.76pp**	**11.91pp** (Kylin)	11 of 12
the same, excluding Kylin	0.84pp	2.93pp	11 of 11

Quote 1.76pp / 11.91pp for anything in Table 3. The favourable reading is also true and is the one to lead with: eleven of twelve land within 2.93pp, and the twelfth is Kylin, whose branch truncation (§4.2.1) is the same defect showing up a second time.

And the estimator is applied entirely outside the range it was validated on. The twelve validation projects span commit-side **82.6–98.3%**; the twenty-one projects that carry the association below span **10.5–78.1%**. The ranges are **disjoint — 0 of 21** applied projects fall inside the validated one. There is a plausible argument that this does not matter, since the error mechanism is tracker-numbering density rather than commit hygiene, and within the twelve the error does not track the commit-side rate ($\rho = -0.16$). That is an argument, not a validation, and we make it as such. **Any claim below whose margin is smaller than 11.91pp is not safe on this estimator.**

Result. Over the 33 projects with a usable denominator, Spearman's ρ between the commit-side rate and the ticket realisation rate is -0.010 (95% CI $[-0.352, +0.335]$, permutation p = **0.958**, 200,000 relabellings). Excluding the three projects whose repositories barely overlap the snapshot era, it is -0.062 on 30 (95% CI $[-0.413, +0.305]$).

	n	**ticket realisation rate**
cleared the 0.80 bar	12	0.04–68.6%, **median 52.6%**
the bar dropped	21	29.5–84.8%, **median 53.2%**

The passing group's median is **0.6pp below** the dropped group's — the bar selects for nothing on this axis (Mann-Whitney $p = 0.94$). **The highest ticket realisation rate in the entire probe — Syncope at 84.8% — belongs to a project the bar rejected at 36.0% commit-side.**

> **What this does and does not establish.** At $n = 33$ this study can detect $|\rho| \geq$ **0.47** at 80% power. The interval admits everything from a moderate negative to a moderate positive association. **A strong positive relationship — the assumption that a high commit-side rate implies a usable ticket-side rate — is ruled out. A null is not established.** Earlier drafts said the two rates are "uncorrelated" and that commit-side "carries essentially no information"; both overstate what $n = 33$ supports and are withdrawn.

And the live measurement points the other way. On the twelve projects where both rates are measured exactly and live — no frozen snapshot, no number-capping — Spearman's ρ is $+0.518$ (95% CI $[-0.080, +0.841]$, permutation $p = 0.088$). That is the opposite sign from the frozen estimate on 33. The two intervals overlap and the pair is not formally contradictory, but **the paper does not have one answer to give here**, and manufacturing one would be worse than saying so. `paper/DIRECTION_TENSION.md` sets out both measurements, the populations they are taken over, and the decomposition that explains the difference — the correlation between commit-side rate and commits-per-ticket is $+0.455$ within the eligible twelve and -0.717 across the 33, so the sign of the composite flips with the population rather than with the estimator.

What the extension converts, and what it does not. §4.2's range was *within-passing variation* and licensed no claim about the projects below the bar. It now licenses a bounded one: across the whole probe, a high commit-side rate does not predict a high ticket-side rate, and the bar selects for neither. **The caveat on Table 1 is not withdrawn** — it remains true of the live measurement, and this is a different estimator against a different snapshot, reported alongside rather than merged into it.

Five projects have no usable rate and are excluded rather than scored zero, which is itself the taxonomy at work. Pinot (14 tracker issues), Dubbo (78) and RocketMQ (384) have denominators

too small to carry a rate. ShardingSphere and SkyWalking have **no project record for their probed key at all** — mode 1 and mode 6, a category rather than a low number (§5.3).

4.4 The measure replicates independently across corpora seven years apart

The commit-side rate is not a novel measure, and its agreement with an independently constructed dataset is the reason to trust the probe. Four of the five projects overlapping SEOSS 33 agree within **3.3pp** on corpora seven years apart: Hadoop 97.13% → 97.8%, Hive 96.34% → 97.0%, HBase 90.06% → 92.5%, ZooKeeper 87.12% → 90.4%.

Flink differed by **24.1pp** on full history (41.98% → 66.0%), and the scope explanation for it has now been tested. Re-probing the **earliest 12,419 commits** — SEOSS's own change-set count — gives **5,214 / 12,419 = 41.9841%** against their **41.98%**: a gap of **+0.0041pp**, where full history gives +24.1pp. **Five of five overlapping projects agree once scope is matched.** The remaining 25,800 commits run at 77.62%, and Flink's by-year series shows why: 0.0% in each of 2010, 2011, 2012 and 2013, 19.4% in 2014, 66.0% in 2015, and 70–88% every year since (`paper/FLINK_TRUNCATION.md`).

The exactness of that match is not a warning sign. The quantity is a deterministic count over a fixed, identically bounded commit range — not an estimate, and with no sampling error. Two correct implementations of the same well-specified count should agree exactly, and a disagreement would indicate a specification difference rather than noise; exact agreement is only suspicious between estimators that *have* sampling error. The two numbers also arrive by independent paths, one transcribed from a published table and one scanned fresh from a clone at a pinned sha. **It confirms scope alignment and nothing more** — in particular it does not validate the ticket-side estimator of §4.3, which is a different measurement with its own error, reported there.

4.5 Convergent invisibility across three channels and two ecosystems

Table 2 assembles what each record channel shows in each ecosystem. **Every cell is a different quantity with a different denominator, and no statistic is computed across them.** They are assembled because they agree in direction while being methodologically independent, and independent agreement is the only kind of evidence a single-corpus critique cannot absorb.

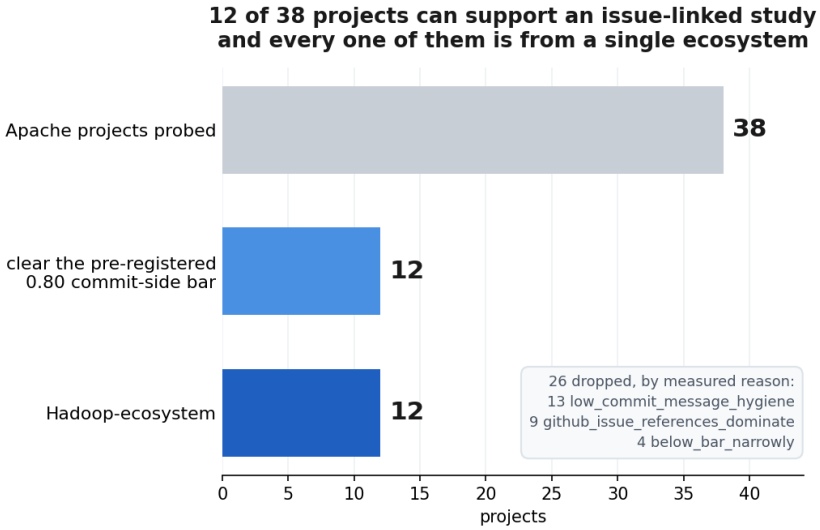
In the TypeScript corpus: **3 of 96** architectural commits mention refactoring in their message (93 are silent); of ten commits whose message claims refactoring, **seven are not architectural**; **7%** link an issue, against 7% for other refactoring and 10% for non-refactoring commits — architectural change is no more likely to be tracked in advance than anything else and slightly less; and **30 of 96 = 31%** pass through a pull request, leaving **69% unreviewed**.

Two cells are absences of different kinds and the distinction is load-bearing. **Apache / pull request is a structural `n/a`**: Hadoop's review record lived in Jira via the githubbot relay during the period studied, so there is nothing to measure and no amount of mining creates it. The nearest Jira-side proxy demonstrates the point by terminating rather than thinning — architectural tickets carrying a CI verdict run 145 of 150 (96.7%) up to 2018, 66 of 130 (50.8%) in 2019–21, and **0 of 43 (0.0%) from 2022**. **Apache / commit message is a gap in this study**, not a property of Apache: the Hadoop architectural corpus was built by AST-level detection and the message-text channel was never instrumented against it, so no figure comparable to the TypeScript column's 3.1% exists here. It is computable in principle and is the most obvious extension of the table.

4.6 The standard sampling frame cannot express the criterion

GHS is the standard sampling tool for MSR studies. Its 2021 publication reports **735,669 repositories**; a record returned by the live API in 2026 carries **35 fields**. (The two figures are of different vintages and are kept apart deliberately: the index has certainly grown since 2021, and the 2021 paper describes 25 characteristics rather than 35.) Only `totalIssues` and `openIssues` touch issues at all, and both are GitHub-issue counts. There is no field for issue-tracker type, external tracker usage, issue–commit linkage, traceability, or commit-message convention; the one adjacent feature, filtering by issue label, is described in the paper itself as still under development.

A researcher sampling with GHS therefore gets stars, commits, contributors and license, and discovers post hoc that 26 of 38 candidates are unusable. That is what happened here, and § 5 is the taxonomy of the ways it happens.



Six operationalisations of the original question, all closed with a named cause — the sampling frame became the finding

Structural review discussion predicts slower resolution	HR 0.69 (p=0.003) → HR 1.10 (p=0.51) *discussion-volume confound*
Blast radius — depended-upon modules invite hesitation	p=0.33 with tier controlled *the connector tier, not centrality*
Maintainer concentration replaces the hand-drawn tier	collapses once module size enters *collinear with module size, rho = −0.86*
Abstraction is harder than relocation	×1.54 [0.97, 2.43], p=0.068, n=319 *split defined post hoc*
Architectural refactoring missed a decade of process improvement	holds to 2019 (+0.192, p=0.021); post-2020 p=0.44 *control arm collapses 394 → 145 → 101*
A portable tier rule generalises	0 of 3 replicate (p=0.210, 1.000, 0.310) *vendor share selects large modules, not thin adapters*

Numbers verbatim from README.md §4. Killing commits: PROJECT_STATE.md §2.

Figure 1: Corpus attrition: 38 probed projects to 12 eligible to a single ecosystem, with the six operationalisations that were tested and closed.

5 Six ways a project cannot support an issue-linked study

Drafted from `paper/eligibility_failure_modes.md` (5179907).

Every mode below was discovered *after* selecting the project, by probing it. None is expressible in any published sampling frame. GHS indexed 735,669 repositories as published in 2021 and exposes 35 fields per record today, of which only `totalIssues` and `openIssues` touch issues, and both are GitHub-issue counts (§2.3). All six were found the same way: by reading commit messages, project by project.

Note on mode 6. `README.md` §5 labels mode 6 "repository migrated across organisations". That framing was **superseded on 2026-07-25** once id→key stability was measured (`PROJECT_STATE.md` §2, 5179907). The current statement is below, and it is the one this paper uses.

#	mode	worked example	evidence
1	**GitHub Issues displaced Jira**	ShardingSphere: **5** Jira citations in 49,111 commits, against 30,746 GitHub-issue references	`paper/traceability_probe.json`
2	**No source repository exists**	RedHat RHBRMS: 86.00% estimate coverage on 2,400 issues — a product/documentation tracker with no code	`estimates_by_org.json`; repo resolution failed
3	**The tracker is downstream of the upstream repo**	kata-containers: **zero** `KATA-` keys in 19,807 commits; the Red Hat Jira tracks a product built from an upstream nobody asks to cite	`paper/intersection.json` (excluded, not scored)
4	**Convention changed mid-history**	spring-batch: `BATCH-` in **45.6%** of 7,035 commits overall, in **0 of the most recent 1,000** (back to 2021-06), and at exactly **0% in every year since 2019** — a single rate averages two regimes	full scan, `scripts/springbatch_recency.py`
5	**Monorepo needs multi-key matching**	Hadoop: **26.2%** single-key → **92.3%** four-key → **97.8%** seven-key, same 28,290 commits	`scripts/citation_rate.py`
6	**The cited key has no project record in the tracker**	Evergreen: commits cite `DEVPROD` (2,785) but the tracker holds only `EVG`. DataLab: commits cite `EPMCDLAB` (3,900) and `DLAB` (3,547); the tracker holds only `DATALAB`	`paper/intersection.json`

5.1 The distinction that makes this a taxonomy rather than a list

Modes 1, 2 and 3 are disqualifying rather than measurable. The project is not a weak candidate; it is not a candidate. Scoring kata-containers as "0% traceable" would place it on the same axis as a project that tries and fails, and would put a number where a category belongs.

Modes 4, 5 and 6 are silent. They produce a plausible-looking low number rather than an error. A single-key probe reads Hadoop as 26.2% and Evergreen as 71.7%, and **nothing in either result signals that the key set is wrong.** Four of the 27 projects in `paper/intersection.json` needed multi-key matching — DATALAB (`DLAB`+`EPMCDLAB`), TRINIDAD (`TRINIDAD`+`ADFFACES`), DAOS (`DAOS`+`CART`), EVG (`EVG`+`DEVPROD`).

This is the practical consequence, and it is the section's point: **project eligibility for issue-linked research cannot be established from metadata.** It requires reading commit messages from the project itself, per project, before any outcome is measured.

5.2 Mode 6 in detail: two mechanisms, only one predictable

Measured across 2,686,282 issues, **project id → key is 1:1**: 0 ids carry more than one key and 0 keys map to more than one id, against **326 ids carrying more than one name**. Keys are stable identifiers *within a tracker snapshot*. So the multi-key requirement is **not** key instability — it has two different causes.

Concurrent siblings — predictable. Several live projects share one repository. Hadoop needs `HADOOP,HDFS,YARN,MAPREDUCE`; IntelDAOS needs `DAOS,CART`; Spring Batch needs `BATCH,BATCHADM`. All of these project records exist in the tracker, so the key set can be enumerated from the Jira instance before touching git.

Superseded records — not predictable. The key cited in commit messages has **no project record in the tracker at all.** DataLab's git history cites `EPMCDLAB` 3,900 times and `DLAB` 3,547 times; the Jira dump contains neither, only `DATALAB` with 1,858 issues. Evergreen cites `DEVPROD` 2,785 times; the dump contains only `EVG`. **Enumerating the tracker's projects cannot recover these keys — they exist only in git.**

Both fail silently, which is the same property as modes 4 and 5: commit-to-ticket matching runs on keys, and the key set cannot be derived from the tracker alone.

A related instance is a repository that resolves elsewhere: `kiegroup/optaplanner` HTTP-redirects to `apache/incubator-kie-optaplanner`, with identical history and PLANNER 1,629 either way. An earlier verdict that this was a misresolved build-config repository was wrong and is corrected in `PROJECT_STATE.md` §7.

5.3 Mode 6 measured across the whole probe, not just its worked examples

The probed key sets were derived empirically from commit messages (§3.2), and the frozen public Jira corpus records every Apache project that existed at its snapshot. Intersecting the two turns mode 6 from an anecdote into a count: **six of the keys this study probes have no project record in that corpus at all**, and five of them are actually cited by commits.

project	key with no tracker record	distinct keys cited under it
pinot	`THIRDEYE`	327
calcite	`OPTIQ`	85
rocketmq	`RIP`	35
skywalking	`SWIP`	11
shardingsphere	`RS`	8
ozone	`OZONE`	0 — probed, never cited

Calcite is the clearest instance: `OPTIQ` was the project's name before it was renamed, its keys are cited 85 distinct times in the repository's history, and **no OPTIQ project exists in the tracker corpus**. Enumerating the tracker's projects cannot recover that key. It exists only in git, and a probe that derived its key set from the tracker — the natural thing to do — would silently miss every commit that cites it.

5.4 One observed instance inside the eligible corpus

The taxonomy is not confined to the projects it disqualified. Among the 12 that passed, **Sqoop's commits cite 790 distinct keys of which 122 have no record in its tracker** (`paper/ticket_coverage.json`) — mode 6 operating inside a project that clears the bar at 82.6%. Its ticket-side rate is computed over the 668 keys that do resolve.

5.5 What a sampling frame would need

None of the six is derivable from repository metadata. Expressing them requires, per candidate: the dominant reference channel (Jira keys against GitHub issues, counted rather than assumed), the tracker's project-key set, the key set actually appearing in commit messages, and the rate computed separately over recent history to expose mode 4. All four are cheap. None is in GHS.

One qualification, because the unqualified claim overstates the gap. We say above that the key set cannot be recovered from the tracker, and that stands: a key cited only in git — `OPTIQ`, `EPMCDLAB`, `DEVPROD` — is not in the tracker to be enumerated. But a substantial literature *does* recover missing issue–commit links without relying on the commit message, beginning with ReLink (Wu et al., ESEC/FSE'11) and continuing since, by learning from time proximity, author identity and textual similarity between report and change (§2.4). Those methods recover **links**, not **key sets**, and they presuppose a project already known to be a candidate — which is the step this taxonomy is about. Modes 4–6 make a project's *measured rate* wrong; link recovery can raise the true rate afterwards. A sampling frame that exposed the four fields above would tell a researcher which projects are worth pointing a link-recovery tool at, which is a different and prior question.

6 Threats to validity

Assembled from `paper/ERA_AUDIT.md` (`e0d76ef`), `paper/ALIASING_HADOOP.md`, `paper/matcher_validation.md`, `SUI_FINDINGS.md`, `paper/numbers.md` and `PROJECT_STATE.md` §7. Several of the items below are things this study got wrong and corrected. **They are stated as the paper's credibility, not as its embarrassments** — a methods paper about record quality that concealed its own record would be self-refuting.

6.1 Construct validity

The measure is 97.5% precise, and two of its failure modes were untestable. Key matching was validated on a seeded 200-commit manual sample across the 12 eligible projects: precision **195 of 200 = 97.5%** (Wilson 95% CI 94.3–98.9%), corpus-weighted 97.7%. The residual is 4 reverts and 1 backport. But two of the five enumerated failure categories **could not have fired**: `version_string` is near-unreachable given a pattern requiring the upper-case key followed by `-` and digits, and `foreign_key` is unreachable for 11 of the 12, because each project is probed with its own key set and only Ozone is multi-key with both keys its own. **The single-key → multi-key result on Hadoop (26.2% → 92.3% → 97.8%) is therefore not validated by that sample**, Hadoop being the only true monorepo in the study and not among the 12 (`paper/matcher_validation.md`).

A key matched from a branch name is counted as a genuine reference. Two mechanisms occur in the sample: merge subjects (`Merge branch 'master' into KNOX-998-Package_Restructuring`) and svn trailers naming a branch (`.../branches/TEZ-1@1471779`). The ticket association is correct even though the commit does not implement the ticket. The five failure categories were fixed before labelling and do not cover this case, so it was counted genuine and is recorded here rather than assigned an invented category.

Ticket-side coverage counts keys that resolve in the tracker. Sqoop cites 790 distinct keys of which 122 have no tracker record, so its ticket-side rate rests on the 668 that resolve (§5.3). This is mode 6 inside a passing project, and it means ticket-side rates are, strictly, coverage of *resolvable* tickets.

The 38-project extension uses a different denominator source, and the validation figure the earlier draft quoted was for a different approximation. §4.3 measures the ticket realisation rate for all 38 projects against the frozen public Jira corpus rather than a second live fetch, aligning numerator and denominator in issue-number space (§3.1.3). **Two approximations are stacked**: number-capping, and substituting a frozen snapshot for the live tracker. The 0.45pp / 2.14pp figure earlier drafts quoted validates only the first. The end-to-end error of the combination Table 3 actually uses is **1.76pp mean / 11.91pp max**, the worst case

being Kylin; excluding Kylin it is 0.84pp mean and **2.93pp max, with 11 of 12 inside 3pp**. **Quote the end-to-end figure.**

And the estimator is applied entirely outside its validated range. The twelve validation projects span commit-side **82.6–98.3%**; the twenty-one that carry the §4.3 association span **10.5–78.1%**. The ranges are **disjoint — 0 of 21** applied projects fall inside the validated one. The error mechanism is tracker-numbering density rather than commit hygiene, and within the twelve the error does not track the commit-side rate ($\rho = -0.16$, $n = 12$), so there is a reason to expect the extrapolation to hold — but it is a reason, not a validation, and **any claim whose margin is smaller than 11.91pp is not safe on this estimator.** The claims that are: the ceiling identity (exact arithmetic, not estimated); the eligibility result (commit-side only); the taxonomy. The claims that are not: any per-project ticket-side comparison in Table 3 closer than ~12pp, and the passing-versus-dropped median gap of 0.6pp, which is far inside the noise and is reported as such.

Four further things can break the estimator, and all four occur in this corpus:

1. **Trackers with gaps.** Number-capping assumes a tracker holding N issues holds approximately the first N numbers. Issues moved or deleted break that, and the estimator then undercounts. This is the largest single validation error in the set and the direction is always downward.

2. **Repositories whose history does not span the snapshot era.** Where a repository has been truncated or re-initialised, almost none of its cited keys fall inside the frozen range and the estimated rate collapses towards zero. That is a *true* statement about what is recoverable from the repository as it now stands, and a badly misleading one if read as a statement about whether the project's tickets were ever worked. Affected projects are flagged in Table 3 with the share of cited keys that postdate the snapshot — **99.72% for Kylin (709 of 711), not 100%**; an earlier draft printed 100%, which contradicts the non-zero numerator that produces Kylin's 0.04%.

This defect is not confined to the frozen arm, and that is the more serious finding. Kylin's pinned commit reaches **968 commits beginning 2022-08-01**, **7.5% of the 12,937 on the repository's refs**, against a tracker of 5,931 issues and a repository that begins 2014-05-13. Its *live* 12.0% is therefore also measured over a four-year window against a twelve-year tracker. Earlier drafts made that 12.0% the paper's flagship example while dismissing the frozen 0.04% as an artefact; **both are the same artefact**, and the example has been moved to Hive (§4.2.1). Three other projects have a pinned branch starting after their repository — Jena (49.0% of refs), Karaf (45.4%), James (89.4%, by eight days) — and none approaches Kylin's severity (`scripts/revision_metrics.py`).

1. **Denominators too small to carry a rate.** Several trackers hold only a handful of issues in the frozen corpus. Projects with fewer than 500 are excluded from the association statistic and shown in the table with the exclusion marked.

2. **Keys with no tracker record at all.** Some probed keys have no project record in the frozen corpus. These are taxonomy modes 1 and 6, not low rates, and they are **excluded rather than scored as 0%** — scoring them would put a number where a category belongs, which is the error §5.1 exists to prevent.

The truncation caveat on Table 1 is not withdrawn. The 12.0–69.0% range from the live measurement remains within-passing variation, computed on the twelve that had already cleared the bar. §4.3 does not extend that measurement; it is a different estimator against a different snapshot, and it is reported alongside rather than merged into it.

"345 of 349 episodes traceable to 323 tickets" — the unit matters. An estimate is a property of a ticket, not an episode; several episodes share a ticket. An earlier draft used 0/345 where 0/323 is correct (`PROJECT_STATE.md` §7).

Two unrelated quantities are both 92.3% and are labelled apart. 92.3%-A is traceability (26,125 / 28,290 Hadoop commits citing a key, four-key probe); 92.3%-B is RefactoringMiner chunk coverage (3,175 / 3,440). Different numerators, denominators, scopes and claims. This paper uses **92.3%-A only** (`paper/numbers.md` §2).

6.2 Internal validity — instruments that decay, and one that terminates

Relevant because it bounds what the Apache column of Table 2 can contain.

instrument	≤2018	2019–21	≥2022
Jira status transitions — architectural tickets reaching `Patch Available`	147 of 150	98 of 130	**6 of 43**
CI verdicts — architectural tickets carrying ≥1 Hadoop-QA comment	145 of 150 (**96.7%**)	66 of 130 (**50.8%**)	**0 of 43 (0.0%)**

The CI channel terminates in 2021. Not decays — there is no architectural ticket after 2021 with a CI verdict in Jira, so no additional mining extends the series, and the mid-window is half-blind rather than merely thinner. The common cause of both is the 2019–20 GitHub migration: work moved to pull requests and the Jira status field went unmaintained.

Two further consequences, both already disclosed in the source memos and generalised by the era audit: status-derived timings are **not comparable across module tiers** at all (cloud connectors drive the Jira workflow in 28.3% of tickets against 86.2% elsewhere, on $n = 13$ connector tickets — thin enough that the proportion is quoted and nothing is leaned on it), and the phase decomposition rests on **6 architectural tickets after 2021**, so any claim that build-versus- merge behaviour *persists* is unsupported.

714 / 714 = 100.0% is true by construction and is never reported as validation. The analysis frame is built from commits that cite tickets, so a ticket with no citing commit cannot enter it and the match rate cannot come out at anything else. What it establishes is that the clock is *applicable* to every ticket in the frame — a statement about the frame, not about coverage of Hadoop's tickets. **The real coverage question is unanswered and is not computed anywhere** (`paper/numbers.md` §4).

6.3 Author identity aliasing does not affect the Hadoop measures

The TypeScript pilot found one developer appearing as two email addresses with opposite workflow habits, which corrupted every author-level statistic in that pass. The obvious question is whether the Hadoop analysis has the same defect.

It does not, and this was measured rather than assumed. `scripts/social_centrality.py` keys on the git author **name** (`%an`), so the two-emails-one-person case is merged by construction. Hadoop's *raw* aliasing is worse than the pilot's — 1,035 emails resolve to 797 identities, and 96 emails span multiple author names covering 27,307 commits, 35.2% of the corpus — but the effect on the derived measures is null: mean `top1_share` 0.1545 by name against 0.1495 by email over 112 modules, **paired Wilcoxon** $p = 0.3651$, no systematic direction. Aliasing severity scales inversely with contributor count, so the same defect is critical in a 53-author repository and irrelevant in an 800-author one; it must be measured per corpus (`paper/ALIASING_HADOOP.md`, `scripts/sui/ALIASING_NOTE.md`).

One difference was never verified. The `reviewer_pool` variable uses a third identity namespace — Jira comment authors — and the two arms read it by different paths (`c["author"].get("name")` for architectural tickets, `c.get("author")` for controls). Immaterial in practice, since `reviewer_pool` is null everywhere (`triage~reviewer_pool` $\rho = 0.0010$, $p = 0.988$), but unverified rather than checked.

6.4 Detector coverage was lost non-randomly, and the obvious bias was tested

A 93%-missing year was found and repaired. The first RefactoringMiner run lost three chunks to hangs, and the loss was *clustered*: 75 of 81 commits in 2024 (93%) were absent, against 2–8% in every other year. Every temporal result from that pass was invalid, including a "zero architectural commits in 2024" that was a measurement hole. A targeted re-run recovered 68 commits and lifted coverage 88% → 93%. **Coverage loss in chunked detector runs must be checked for clustering, not merely reported as a percentage.**

The natural worry that loss is biased toward large commits was tested and rejected. Within the detector's actual scope (`root..HEAD`, 1,198 commits; 1,057 analysed, 141 lost), lost commits are *smaller* — median churn 14 lines against 32, $p = 0.0022$ — with no enrichment in the top churn decile (9.6% against 12.8%, $p = 0.23$). The mechanism is structural: the watchdog kills a whole chunk, so ~30 commits die because one of them hung, regardless of their own size.

A first attempt at that test was wrong and is kept as a caution. It compared against `git log --all`, so the "missing" set was contaminated with 930 commits on other branches that were never in scope, and it produced the opposite, spurious conclusion that lost commits were larger. The comparison set for a coverage-bias test must be exactly the range the tool was asked to analyse.

6.5 Single-rater limits, and the one measure that escapes them

Inter-rater agreement cannot be computed by one person, so no measure requiring manual coding carries a claim in this paper. Two instruments are affected and both are reported **as findings about the instrument**, never as instruments the paper's claims rest on:

- the codebook's keyword rule for structural review discussion — **≈25% precise** (5 of 20 flagged comments genuinely structural), ≈5% miss (1 of 20 unflagged), on a 40-comment single-rater pass, with no κ;
- the TypeScript commit-message rule — **recall 3 of 96 = 3.1%**, **precision 3 of 10 = 30%**.

Their role is to establish that keyword-based detection over-counts roughly fourfold and finds 3% of the work, which is a validity result about a method other studies use.

The exception is argued rather than asserted. The 200-commit matcher check is a mechanical determination against a written rule with categories fixed before labelling, not a coded judgment where the construct lives in the rater's head. It is made **auditable rather than agreed**: `paper/matcher_sample.json` carries all 200 messages and `paper/matcher_labels.json` every label, so any reader can re-check the sample without redrawing it.

The architectural-episode gold set has no second rater and is therefore not used to support any claim here.

One coincidence in the rater pilot is disclosed because it looks like tuning and we cannot prove it is not. `paper/LLM_RATER_PILOT.md` reports that re-labelling six comments under the opposite ordering of two codebook rules moves the attainable κ ceiling from 0.491 to

0.840. Five of those six are in the flagged bucket, and **five is exactly the reclassification count that maximises the ceiling over every possible count**: four gives 0.765, six gives 0.837, seven gives 0.833. The six were selected by a stated semantic criterion, they are individually defensible, and the finding is robust at 0.833–0.840 across plus or minus two comments — but the criterion was applied by the same party that reported the resulting number, and it landed on the global maximum. Stated here rather than left for a reader to find (`audit/JUDGMENTS.md` §3).

6.6 Definitions fixed after seeing data, and thresholds held when they hurt

The abstraction-versus-relocation split was defined post hoc, after seeing the triage numbers it was later used to analyse. Three definitions — any-rule, pure-vs-pure, continuous share — give the same answer, which is reassuring and not a substitute for pre-registration. Fully adjusted the effect is $\times 1.54$ $[0.97, 2.43]$, $p = 0.068$, $n = 319$.

The estimate conclusion was drawn at the wrong level of aggregation and is self-corrected. "Effort estimates absent in Apache (0%)" was replaced by three numbers that travel together: **2.557%** Apache-wide (25,949 / 1,014,926), **1.449%** in the Hadoop corpus (713 / 49,201), **0 of 323** architectural tickets — expected 4.7, P $\approx$ 0.009 under a naive binomial. Architectural tickets are not a random draw and the non-randomness could run either way, so the deficit is suggestive on thin evidence, not an established effect.

A threshold was held during a failed replication when lowering it would have rescued two projects. The vendor-share tier rule was frozen at $\geq$0.40 before any held-out outcome was observed. HBase (maximum share 0.33) and Phoenix flagged nothing at that cut, so neither could be tested. **The threshold was not lowered.** The rule then failed 0 of 3 where it could be tested. That the fix was available and declined is recorded deliberately (`replication/REPLICATION.md`, `PROJECT_STATE.md` §2 row 07-25).

The eligibility bar was likewise fixed before any project was cloned and never moved. Lowering it to 60% would have added nine projects and widened the family.

6.7 External validity

One mined project, one ecosystem. All 12 eligible projects are Hadoop-ecosystem, and depth beyond Hadoop was not reachable. The corpus bound is therefore a finding and a limit at once: results about Apache/Jira/Java traceability generalise to Apache/Jira/Java.

At 12 projects the design is not powered, whatever the corpus size. Effective n is capped at P/ICC by between-project correlation — a ceiling of 600 at ICC 0.02 against the 769 the effect needs. The direction of the bias is bad: these twelve share contributors, committers, review norms and in several cases build and CI infrastructure, so this sample's ICC is higher than a random sample's. **The ICC itself is unmeasured, and was deliberately not estimated from the four available clusters**, because between-cluster variance is not estimable at $n = 4$ and the estimate must come from the pooled study as a first stage (`replication/CORPUS_FEASIBILITY.md`).

The TypeScript column is one repository, one company, and exploratory. `BearStudio/start-ui-web`, 1,199 commits, 93% detector coverage, $n = 28$ **architectural PRs**. `SUI_FINDINGS.md` is marked EXPLORATORY throughout and every p-value in it is uncorrected and hypothesis-generating. **Only counts and proportions from it are used here** — 3 of 96, 7%, 30 of 96 — never a test. The repository is also a starter template whose product is tracking the frontend stack, so migration-driven churn is unusually large (architectural commits co-occurring with a dependency-manifest change are 13$\times$ larger).

PR routing there is not arbitrary, which bounds what the 31% means: architectural commits that go through a PR are five times larger and four times more abstraction-heavy than those pushed to main. The 69%-unreviewed figure is a statement about that repository's workflow, not a general rate.

The Apache commit-message cell of Table 2 is empty because this study did not instrument it. The Hadoop architectural corpus was built by AST-level detection and never compared against message text, so no recall figure comparable to the TypeScript column's 3.1% exists. It is computable in principle and is the most obvious extension. Note also that the 25%-precision codebook figure measures **review comments**, a fourth channel, and is not a commit-message result (`paper/table2_visibility.md`).

6.8 Provenance of the work itself

Built by one person in three working sessions alongside full-time employment, with a committed history spanning 19–25 July 2026 and this completion pass on 30 July. That is what set corpus depth at one project and left no second rater; it is stated because it explains the shape of the limitations above rather than excusing them.

The Jira caches are frozen with no rebuild script, deliberately. Jira is live and a re-fetch returns different state, so a rebuild script would imply a reproducibility that does not exist. The archive is 2,491 files with a per-file SHA-256 manifest (`scripts/freeze_caches.py`, `deposit/MANIFEST-v1.md`).

The dated working logs are left unrewritten (`SLICE_LOG.md`, `worksheet.md`, `advisor_brief.md`, `SUI_ADVISOR_BRIEF.md`), so the record of what was believed when stays intact and can be checked against what is claimed now.

Held-out discipline is mechanical, not merely intended. Seven projects contribute coverage counts only; `paper/ticket_coverage.json` records the Jira fields screened out, and the matcher validation requests `%H`, `%s` and `%b` from git and nothing else, so it *cannot* compute a duration. HBase and Phoenix were dropped from the held-out corpus on 2026-07-30 and quarantined, because their committed artifacts are one subtraction from per-ticket latency even though no outcome was ever observed for either (`spent/README.md`).

One published figure is not reproducible and is never quoted beside a per-project number. The public Jira dataset reports 1,822 projects; counting final-state keys gives **1,276** and implementing the dataset's own name-union method gives **2,506**. 326 project ids carry more than one name and 0 keys do, so rename inflation explains part of the gap and not all of it. The three figures travel together or not at all.

7 Discussion

7.1 For researchers designing an issue-anchored study

Project eligibility cannot be established from metadata. Every one of the six failure modes in §5 was found by reading commit messages from the project itself, after the project had been selected. Three of them are silent: they yield a plausible low number rather than an error, and nothing in the result signals that the key set is wrong. A researcher who samples on stars, contributors and language, then computes a linkage rate, will get a number for every candidate and will not be told which of those numbers mean anything.

Measure the side of the rate your design actually samples on, and report commits per ticket beside it. A commit-side rate answers "if I start from a commit, can I find its

ticket?". A design that starts from *tickets* needs the ticket realisation rate, and the first does not imply the second — Hive is 97.0% one way and 55.8% the other. But the more useful advice is the ceiling (§3.1.4): **below one commit per ticket the two rates are not commensurable at all**, and every project in our eligible corpus is below it except Ranger and Knox. Report `commits / tickets`; it costs nothing, it bounds the ticket-side rate before any measurement, and no published linkage rate carries it.

What our own extension does and does not license. Across the whole probe we find no detectable association between the two rates ($\rho = -0.010$, $n = 33$, 95% CI $[-0.35, +0.34]$). That rules out a strong positive relationship — the assumption a selection bar encodes — but at 80% power this study detects only $|\rho| \geq 0.47$, so it does not establish a null. On the twelve projects measured exactly the same correlation is $+0.518$, and we report the disagreement rather than resolving it (`paper/DIRECTION_TENSION.md`).

Report the bar, the attrition, and the rejected candidates. The dropped projects are the informative part. That 26 of 38 Apache candidates are unusable, and that the 12 survivors are one ecosystem, is a fact about the population that a paper reporting only its final corpus cannot convey — and it is a fact each such paper independently rediscovers and does not write down.

7.2 For dataset and sampling-frame builders

The fields that decide usability are absent from the standard frame. GHS indexed 735,669 repositories as published in 2021 and exposes 35 fields per record today; only `totalIssues` and `openIssues` touch issues and both are GitHub-issue counts. There is no tracker type, no external tracker, no issue–commit linkage, no commit-message convention.

Four fields would close most of the gap, and all four are cheap to compute from a shallow clone and a tracker listing:

1. **the dominant reference channel** — Jira-key citations against GitHub-issue references, *counted*, not assumed;
2. **the tracker's project-key set**, which catches concurrent siblings (mode 5);
3. **the key set actually appearing in commit messages**, which is the only thing that catches superseded records (mode 6) — those keys exist in git and nowhere else;
4. **the rate recomputed over recent history**, which exposes a convention that changed mid-history (mode 4).

What the four fields would cost. All four come from a `--filter=tree:0` bare clone plus one tracker listing. On this corpus those 38 clones came to **228 MB, a mean of 6.0 MB per project**, against the 4.3 GB the original working-tree sweep took — a 19x reduction. Fields 1, 3 and 4 need only `git log` over commit messages — no trees, no blobs, no working tree. Field 2 is one paginated tracker call for issue keys. For a frame that already crawls repository metadata at the scale of 735,669 repositories, the marginal cost is the clone, and the clone is the cheapest kind there is. We are not claiming it is free at that scale; we are claiming it is the same order as what GHS already does per repository, and that nobody has to guess.

This study spent 38 clones and 4.3 GB to keep 12. That cost is paid again by every group that attempts the same kind of corpus, and none of it is recoverable from published metadata.

7.3 For the field

If architectural change is invisible in three record channels across two ecosystems, then a literature built on those records is measuring the minority of architectural work that someone chose to

expose. The obvious hope is that the visible minority is a random sample of the whole. **In the one corpus where we could test that, it is not.** In the TypeScript corpus, architectural commits routed through a pull request are five times larger by churn and four times more abstraction-heavy than those pushed directly to main. The 31% that are visible are systematically the big, abstraction-heavy ones, and the 69% that are not are systematically the small, relocation-shaped ones.

That is a selection effect on the dependent variable, and it points the same way in every study that mines only what was recorded: architectural work will look larger, more deliberate and more discussed than it is.

7.4 What follows from the corpus limit, without softening it

Twelve projects is not enough, and more projects would not fix it. Between-project correlation caps the effective sample at **P/ICC** whatever the corpus size: at P = 12 and ICC = 0.02 the ceiling is 600 against the 769 independent tickets the effect needed at 80% power. **The ICC is unmeasured**, and was deliberately not estimated from the four clusters available — between-cluster variance has 3 degrees of freedom at $n = 4$, and the decision turns on the difference between 0.015 and 0.02, which four clusters cannot resolve. It must come from the pooled study as a first stage. The direction of the bias is also bad: these twelve share contributors, committers, review norms and in several cases build and CI infrastructure, so this sample's ICC exceeds a random sample's, and higher ICC is what makes the study impossible.

Three ways out, with their costs. **Lower the bar and model the measurement error** — at 60% another nine projects qualify, at the cost of a biased sample of tickets *within* each project, which §7.3 shows is real rather than hypothetical. **This is the one to avoid drifting into silently. Change the outcome so it needs no tickets** — anything computed purely from git dissolves both the traceability filter and the ecosystem clustering, at the cost of the creation-to-first-commit clock and with it the ability to measure waiting at all. **Accept ecosystem-bounded scope and say so** — register as a study of one ecosystem, twelve projects, and state the boundary as a finding rather than discovering it in review. The second is the strongest study and the largest rebuild; the third is honest and immediately actionable.

7.5 What this paper does not claim

It does not claim that Jira-citing projects are better engineered, or that the projects on GitHub Issues are less traceable in any sense that matters to their own maintainers. The convention this research needs is not a quality signal; it is a research affordance, and its decline is a cost to researchers rather than to the projects.

It does not propose the successor design that the corpus limits point towards. That is a separate registered report with its own feasibility gates, and its novelty margin against an active architectural-debt time-to-fix literature is recorded in the replication package as **not assessed**.

And it does not offer a fix. The six modes are diagnosable, not repairable: a project whose tracker is downstream of its repository will not start citing keys because a researcher would like it to.

8 Acknowledgements and disclosures

8.1 Data and tools

This study is built on artifacts other people made public. The Apache Software Foundation publishes the issue trackers and repositories the whole corpus is drawn from. Lloyd Montgomery,

Clara Lüders and Walid Maalej deposited the Public Jira Dataset (Zenodo 15719919, CC BY 4.0), which supplies every estimate-coverage base rate and every frozen ticket denominator in Table 3. Michael Rath and Patrick Mäder's SEOSS 33 provides the independent measurement this study's probe is validated against, and the agreement between them at matched scope (Section 4.4) is the strongest external check the work has. Nikolaos Tsantalis and the RefactoringMiner contributors built the detector. None of them is responsible for what is done with their work here.

8.2 Use of AI tools

Declared in full, because a paper about the trustworthiness of records should be candid about how its own were produced.

Large language model assistants (Anthropic's Claude) were used across parts of this work. Their role was:

- **writing and running analysis code.** Every script in `scripts/` that produced a number in this paper was drafted with assistance. The scripts are committed, the artifacts they emit are committed, and each is re-runnable — that is what makes the assistance auditable rather than something the reader must take on trust.
- **drafting prose.** The manuscript sections were drafted with assistance from the author's outlines, findings and decisions, and revised by the author.
- **literature verification.** Citations were checked against source texts — authors, venue, year, identifier, and whether the source says what is attributed to it. The record of those checks is in the replication package.
- **an adversarial audit of this work's own numbers.** An independent AI-conducted pass re-derived the load-bearing results from primary artifacts with separately written code, exhaustively enumerated the κ bounds, verified all 2,491 frozen cache files against their manifest, and reported what it found — including several errors that were then corrected. Its report is in the replication package under `audit/`, unedited.

What the tools did not do. The research questions, the study design, the pre-registered 0.80 bar and its commitment before any project was cloned, the decision to hold that bar and the vendor-share threshold when moving them would have helped, the six retractions, and the judgment of what the results mean are the author's. **The author takes full responsibility for all content, including any error a tool introduced and the author did not catch.**

A line-by-line verification checklist for this section, separating what is observable in the repository's record from what is inferred about the sessions that predate it, is at `paper/DISCLOSURE_VERIFICATION.md`.

Errors that assistance introduced and review caught are recorded rather than quietly fixed, because they bound how much the audit trail is worth: a median computed as the upper-middle value at even n; a validation figure quoted for a different approximation than the one in use; a provenance checker that could not fail, and a first rebuild of it that reproduced the same defect by another route. Each is in `paper/REVISION_LOG.md` with what it changed.

8.3 A separate matter: the LLM used as a rater

Section 6.5 reports a pilot in which a language model applied this study's codebook as a second rater. **That is an object of study here, not a method the paper's claims rest on**, and it is reported with its contamination disclosed and its conclusions bounded. No claim in this paper depends on it.

8.4 Funding and competing interests

No funding was received. The author declares no competing interests. The work was carried out independently, alongside full-time employment, which is stated in Section 6.8 because it explains the shape of the study's limitations.

Tables

12 of 38 projects clear the pre-registered 0.80 commit-side bar. All 12 are Hadoop-ecosystem.

Eligible (12)

Table 1: Corpus eligibility across the 38 probed projects, both traceability channels, with commits per ticket, the arithmetic ceiling of Section 3.1.4 and the share of it reached.

#	project	Jira key(s)	HEAD	commits scanned	single-key	multi-key	tickets total	commits/ ticket	ceiling	tickets cited	ticket-side	TRR/ ceiling
1	ozone †	`HDDS,OZONE`	`4f5ae5454f`	10,962	98.3%	**98.3%**	15,909	0.69	67.7%	10,162	**63.9%**	0.94
2	tez †	`TEZ`	`dd8137f4c1`	3,123	97.4%	**97.4%**	4,724	0.66	64.4%	2,774	**58.7%**	0.91
3	hive	`HIVE`	`6730aadd18`	18,213	97.0%	**97.0%**	29,635	0.61	59.6%	16,529	**55.8%**	0.94
4	hbase ‡	`HBASE`	`4d417aacea`	21,220	92.5%	**92.5%**	30,063	0.71	65.3%	16,704	**55.6%**	0.85
5	phoenix ‡	`PHOENIX`	`1b55fb4b33`	4,264	92.0%	**92.0%**	7,931	0.54	49.5%	3,277	**41.3%**	0.84
6	zookeeper †	`ZOOKEEPER`	`53a78e36f9`	2,718	90.4%	**90.4%**	4,875	0.56	50.4%	2,291	**47.0%**	0.93
7	ranger †	`RANGER`	`f133c21389`	5,390	86.3%	**86.3%**	5,699	0.95	81.6%	3,725	**65.4%**	0.80
8	oozie †	`OOZIE`	`8bdac8be4f`	2,412	85.2%	**85.2%**	3,727	0.65	55.1%	1,851	**49.7%**	0.90
9	knox †	`KNOX`	`6bdf64cdfd`	3,194	84.3%	**84.3%**	3,391	0.94	79.4%	2,340	**69.0%**	0.87
10	drill	`DRILL`	`86e9b82f16`	4,594	84.2%	**84.2%**	8,507	0.54	45.4%	3,676	**43.2%**	0.95
11	kylin	`KYLIN`	`b5b94b51ab`	968	83.9%	**83.9%**	5,931	0.16	13.7%	709	**12.0%**	0.87
12	sqoop †	`SQOOP`	`f8beae32a0`	969	82.6%	**82.6%**	3,152	0.31	25.4%	668	**21.2%**	0.84

† held-out corpus: coverage and existence counts only, no outcome ever observed (PROJECT_STATE.md §3). ‡ quarantined 2026-07-30, dropped from the held-out corpus; coverage retained, outcome inputs sequestered in spent/ (spent/README.md).

Dropped (26)

Table 2: Corpus eligibility across the 38 probed projects, both traceability channels, with commits per ticket, the arithmetic ceiling of Section 3.1.4 and the share of it reached (continued).

#	project	HEAD	commits	multi-key	drop reason (measured)
13	atlas	b45faecc96	4,140	78.1%	below_bar_narrowly
14	james-project	f4b8d37110	17,374	74.8%	below_bar_narrowly
15	flume	9acf154361	2,084	73.9%	below_bar_narrowly
16	karaf	d3a178529f	10,058	71.8%	below_bar_narrowly
17	calcite	8447eda2f2	6,746	67.7%	low_commit_message_hygiene
18	flink	1ea8cb0e4e	38,219	66.0%	low_commit_message_hygiene
19	hudi	26b1ebc4e9	7,659	65.2%	github_issue_references_dominate
20	oodt	f3dda2591b	2,251	62.0%	low_commit_message_hygiene
21	zeppelin	597652e4c2	5,708	60.7%	github_issue_references_dominate
22	servicecomb-java-chassis	ac4a6991e6	4,668	56.2%	low_commit_message_hygiene
23	tika	379b32246f	10,543	54.8%	low_commit_message_hygiene
24	struts	73f17c1be7	8,353	54.1%	low_commit_message_hygiene
25	storm	cca249f181	11,871	46.7%	low_commit_message_hygiene
26	accumulo	5c7d96deb6	15,069	43.0%	low_commit_message_hygiene
27	cxf	1695c05def	19,763	37.5%	low_commit_message_hygiene
28	syncope	21d513d1c3	9,422	36.0%	low_commit_message_hygiene
29	tomee	a3e8806a9d	16,228	32.2%	low_commit_message_hygiene
30	wicket	34ef6da88d	22,260	30.9%	low_commit_message_hygiene
31	parquet-java	83c2c80d49	2,990	29.1%	github_issue_references_dominate
32	jena	fbe7bfcb1e	13,091	24.4%	low_commit_message_hygiene
33	dubbo	eb1d8abaeb	8,892	11.0%	github_issue_references_dominate
34	helix	0902505fda	4,926	10.5%	github_issue_references_dominate
35	rocketmq	b37e2bbacd	9,165	5.7%	github_issue_references_dominate
36	pinot	2bcbdfed0a	15,639	5.2%	github_issue_references_dominate
37	skywalking	dddc3b51be	8,577	0.3%	github_issue_references_dominate
38	shardingsphere	b6ff1fb56d	49,111	0.0%	github_issue_references_dominate

Caption

The two traceability channels diverge, and the one the literature publishes is not the one an issue-linked study needs. The commit-side rate — what fraction of commits cite a ticket — is what SEOSS 33 reports and what selection criteria use. The ticket-side rate — what fraction of tickets ever receive a citing commit — is what a ticket-anchored study depends on. **Kylin cites a ticket in 83.9% of its commits while only 12.0% of its tickets are ever touched by one** — it clears the bar comfortably and leaves seven of every eight tickets with no commit at all. Across the 12 eligible projects the commit-side rate runs 82.6–98.3% while the ticket-side rate runs **12.0%–69.0%, with none above 69.0%**.

The ceiling, and why it reframes the divergence. A commit that cites a ticket contributes at most one NEW distinct ticket to the ticket-side numerator, so the rate cannot exceed `commit-side x commits / tickets` (method §3.1.4). That bound BINDS for all twelve: it runs 13.7% (kylin) to 81.6% (ranger), and every project reaches 80–95% of it. The ticket-side spread is therefore almost entirely a spread in commits per ticket, not in citation discipline — `TRR/ceiling` has a median of 0.89 and a range of only 1.19x across the twelve, against a 5.8x spread in the rate itself.

Truncation caveat, load-bearing. Ticket-side coverage was computed only for the 12 projects that had already cleared the commit-side bar. The 12.0%–69.0% range is therefore *within-passing variation* and licenses no claim about the 26 dropped projects, whose ticket-side rates were never measured, nor about any correlation between the two channels in general.

The caveat above is not withdrawn by Table 3. `paper/table3_ticket_side.md` computes a ticket realisation rate for every probed project, including the dropped ones, but it does so with a *different denominator source* – the frozen public Jira corpus rather than the live tracker read for this table – and a different alignment (issue-number rather than date). It is reported alongside this table, never merged into it.

Drop reasons are measured, not inferred. The probe counts GitHub-issue references (`#NNN`, `GH-NNN`) alongside Jira keys, so `github_issue_references_dominate` means the GitHub count exceeds the Jira count in that repository. `below_bar_narrowly` ($\geq$70%) and `low_commit_message_hygiene` are threshold labels on the measured rate.

No timing column exists in this table and none may be added for a † row. Every quantity above is a coverage or existence count, which is what the held-out rule permits (`PROJECT_STATE.md` §3).

Sources: `scripts/traceability_probe.py` → `paper/traceability_probe.json`; `scripts/ticket_coverage.py` → `paper/ticket_coverage.json`. Generated by `scripts/make_table1.py`.

Rows are **record channels** — the three places an empirical software engineering study can look for evidence that architectural work happened. Columns are **ecosystems**. Each cell reports how much architectural change is visible in that channel, with its denominator.

Table 2: Three-channel visibility of architectural change across two ecosystems. Every cell is a different quantity with its own denominator and no statistic is computed across them.

channel	Apache / Jira / Java (Hadoop)	start-ui-web / GitHub / TypeScript
commit message	**not measured** — no commit-message keyword instrument was built for this corpus. Architectural episodes were detected from ASTs (RefactoringMiner over 8,919 commits), never from message text, so no recall or precision figure against message text exists. See the note below: the 25%-precision codebook figure is a *review-comment* instrument and does not belong in this cell.	**recall 3 of 96 = 3.1%** of architectural commits mention refactoring; **93 of 96 are silent**. **Precision 3 of 10 = 30%** — of 10 commits whose message claims refactoring, 7 are not architectural. Denominators: 96 architectural commits (recall), 10 refactoring-claiming messages (precision).
issue tracker	**commit-side 82.6%–98.3%** of commits cite a ticket, across 12 eligible projects; denominator = commits scanned per project, 968--21,220. **Ticket-side 12.0%–69.0%** of tickets ever receive a citing commit, same 12 projects; denominator = tickets in the tracker per project, 3,152--30,063. Kylin: **83.9%** against **12.0%**.	**7 of 96 = 7%** of architectural commits link an issue. Denominator = 96 architectural commits. For comparison in the same corpus: other refactoring 7%, non-refactoring 10% — architectural change is no more likely to be tracked in advance than anything else, and slightly less.
pull request	**n/a** — pre-migration Jira workflow. Hadoop's review record lived in Jira via the githubbot relay, not in pull requests, so there is no PR-level architectural-visibility measurement to make. The Jira-side proxy for it does not survive the 2019–20 GitHub migration either: architectural tickets carrying a CI verdict run **145 of 150 (96.7%) ≤2018 → 66 of 130 (50.8%) 2019–21 → 0 of 43 (0.0%) ≥2022** (`paper/ERA_AUDIT.md`). The channel is absent by workflow, not merely thin.	**30 of 96 = 31%** of architectural commits are visible at PR level; **66 of 96 = 69% never pass through code review at all**. Denominator = 96 architectural commits.

Caption

This table demonstrates convergent invisibility across non-comparable measures. It is not a single metric measured six times.

Every cell is a *different quantity* with a *different denominator*, and that is the point of the table rather than a defect in it. A recall rate over architectural commits, a citation rate over commits, a coverage rate over tickets, and a review-exposure rate over commits are four distinct operationalisations of "visible", answering four distinct questions, on two corpora that share no scale, no language, no governance model and no tracker. They are assembled here because they **agree in direction while being methodologically independent** — and independent agreement is the only kind of evidence a single-corpus critique cannot absorb.

Consequently: **the cells are not normalised to a common scale and no summary statistic is computed across them.** Any average, ratio or aggregate over this table would be meaningless, and a reader who wants "the overall invisibility rate" is asking for a number that does not exist. The claim the table supports is qualitative in form and quantitative in each cell: *in every channel measured, in both ecosystems, most architectural change leaves no usable record.*

The strongest single measurement is the issue-tracker row of the Apache column — the commit-side/ticket-side divergence, 12 projects, pre-registered bar, both channels measured together (Table 1). It is reported as evidence for the broader claim rather than as the claim itself, because a single-dataset critique is dismissible as an Apache artifact and a two-ecosystem, three-channel result is not.

Two cells that are absences, and what kind of absence each is

Apache / pull request is a structural `n/a`, not a missing measurement. Hadoop did not use pull requests as its review record during the period studied. There is nothing to measure, and no amount of additional mining creates it. The CI-verdict series is given as the nearest available proxy precisely to show that it terminates: 0 of 43 after 2021 is absence, not sparsity.

Apache / commit message is a gap in this study, not a property of Apache. The Hadoop architectural corpus was built by AST-level detection, and the message-text channel was never instrumented against it. A recall figure comparable to the TypeScript column's 3.1% is computable in principle and is not computed here. Stated as a limitation in `paper/manuscript/threats.md`; it is the most obvious extension of this table.

Note — where the 25% codebook figure actually belongs

`codebook_results.md` and `results_dossier.md` §11 report a keyword rule that is **≈25% precise** (5 of 20 flagged genuinely structural) with a **≈5% miss rate** (1 of 20 unflagged), on a **40-comment single-rater sample**. That instrument measures **review comments** — `codebook.md` fixes the unit of coding as "one review comment on the episode's PR, as relayed into Jira" — and it is a fourth channel, review discussion, not the commit-message channel. It is therefore not placed in the Apache commit-message cell above.

It is also reported **as a finding about the instrument, never as an instrument this paper's claims rest on**: it is a single-rater pass with no κ, and `README.md` §8 records that no measure requiring manual coding is currently defensible in this repository. Its role in the paper is to establish that keyword-based detection of structural discussion over-counts roughly fourfold, which is a validity result about a method other studies use — the same role the TypeScript column's 30% precision plays for commit messages.

Sources

Table 3: Three-channel visibility of architectural change across two ecosystems (continued).

cell	source	script / artifact
Apache, issue tracker (commit-side)	`paper/table1_eligibility.md`, `paper/traceability_probe.json`	`scripts/traceability_probe.py da74465`
Apache, issue tracker (ticket-side)	`paper/table1_eligibility.md`, `paper/ticket_coverage.json`	`scripts/ticket_coverage.py e0d76ef`
Apache, pull request (CI verdict series)	`paper/ERA_AUDIT.md`	`scripts/ci_rework.py 47456b1`
TypeScript, commit message	`SUI_FINDINGS.md` "Secondary findings 1"	`scripts/sui/investigate3.py`
TypeScript, issue tracker	`SUI_FINDINGS.md` "Secondary findings 2"	`scripts/sui/investigate3.py`
TypeScript, pull request	`SUI_FINDINGS.md` robustness checks 5–6	`scripts/sui/selection_bias.py`
codebook note	`codebook_results.md`, `results_dossier.md` §11, `codebook.md`	`scripts/pr_review_signal.py de657c3`

Corpus bounds that travel with the TypeScript column. One repository, one company: `BearStudio/start-ui-web`, 1,199 commits 2019-12 → 2026-07, 93% detector coverage, $n = 28$ **architectural PRs**. `SUI_FINDINGS.md` is marked EXPLORATORY throughout and every p-value in it is uncorrected and hypothesis-generating. The three cells used here are **counts and proportions, not tests**, which is why they are usable in a results table where the pilot's p-values are not.

Denominators come from the frozen public Jira corpus, not from a second live fetch; numerator and denominator are aligned in issue-number space (method §3.1.3).

Two validation figures, and the smaller one does not apply to this table. Against the 12 projects with a published exact rate, number-capping alone is accurate to 0.45pp mean / 2.14pp max. But this table also substitutes a frozen snapshot for the live tracker, and the END-TO-END error of that combination is **1.76pp mean / 11.91pp max** (kylin), with 11 of 12 within 3pp (worst of those 2.93pp). **Quote the end-to-end figure for anything in this table.**

The estimator is applied entirely outside the range it was validated on. The 12 validation projects span commit-side 82.6–98.3%; the 21 projects that carry the association below span 10.5–78.1%. The two ranges are **DISJOINT**: 0 of 21 applied projects fall inside the validated range. Any claim resting on this table whose margin is smaller than 11.91pp is not safe on this estimator.

Ceiling and fill. A commit citing a ticket adds at most one NEW distinct ticket, so TRR $\leq$ CSR $\times$ commits / tickets (method §3.1.4). `ceiling` is that bound; `TRR/ceiling` the share of it reached. A ceiling above 100% does not bind — it means the repository has more citing commits than the snapshot has tickets, which is what a 2026 commit window against an older tracker produces.

Table 3: The ticket realisation rate across all 38 probed projects, with the note column recording every reason a row should not be read at face value.

#	project	bar	commit-side	commits/ ticket	ceiling	ticket realisation	TRR/ ceiling	cited ≤ N	tracker N	note
1	ozone	pass	98.3%	1.79	176.0%	**66.1%**	0.38	4,047	6,121	60.18% of cited keys postdate the snapshot
2	tez	pass	97.4%	0.72	70.0%	**58.6%**	0.84	2,548	4,347	—
3	hive	pass	97.0%	0.71	68.7%	**56.1%**	0.82	14,427	25,731	—
4	hbase	pass	92.5%	0.80	74.3%	**55.5%**	0.75	14,671	26,421	—
5	phoenix	pass	92.0%	0.65	59.5%	**40.7%**	0.68	2,684	6,593	—
6	zookeeper	pass	90.4%	0.64	57.7%	**46.2%**	0.80	1,969	4,263	—
7	ranger	pass	86.3%	1.51	130.4%	**68.3%**	0.52	2,436	3,567	—
8	oozie	pass	85.2%	0.66	56.3%	**49.7%**	0.88	1,811	3,646	—
9	knox	pass	84.3%	1.19	100.0%	**68.6%**	0.69	1,849	2,694	—
10	drill	pass	84.2%	0.57	47.9%	**41.6%**	0.87	3,355	8,069	—
11	kylin	pass	83.9%	0.19	16.3%	**0.04%**	0.00	2	4,989	99.72% of cited keys postdate the snapshot
12	sqoop	pass	82.6%	0.31	25.4%	**21.0%**	0.83	663	3,152	—
13	atlas	drop	78.1%	0.93	72.4%	**54.2%**	0.75	2,420	4,469	—
14	james-project	drop	74.8%	4.32	323.4%	**42.7%**	0.13	1,717	4,018	—
15	flume	drop	73.9%	0.62	45.6%	**42.4%**	0.93	1,434	3,380	—
16	karaf	drop	71.8%	0.74	53.4%	**29.5%**	0.55	3,990	13,537	—
17	calcite	drop	67.7%	1.36	91.9%	**53.2%**	0.58	2,645	4,968	—
18	flink	drop	66.0%	1.50	99.0%	**53.2%**	0.54	13,552	25,492	—
19	hudi	drop	65.2%	2.41	157.5%	**46.5%**	0.29	1,474	3,172	68.28% of cited keys postdate the snapshot
20	oodt	drop	62.0%	2.16	134.1%	**61.5%**	0.46	640	1,040	—
21	zeppelin	drop	60.7%	1.02	61.7%	**50.2%**	0.81	2,816	5,614	—
22	servicecomb-java-chassis	drop	56.2%	2.01	112.7%	**39.7%**	0.35	924	2,326	—
23	tika	drop	54.8%	2.90	159.0%	**58.4%**	0.37	2,123	3,633	—
24	struts	drop	54.1%	1.64	88.7%	**36.0%**	0.41	1,832	5,093	—
25	storm	drop	46.7%	3.16	147.2%	**55.7%**	0.38	2,096	3,762	—
26	accumulo	drop	43.0%	3.18	136.5%	**61.1%**	0.45	2,899	4,745	—
27	cxf	drop	37.5%	2.32	86.8%	**53.9%**	0.62	4,598	8,531	—
28	syncope	drop	36.0%	5.71	205.5%	**84.8%**	0.41	1,399	1,650	—
29	tomee	drop	32.2%	3.00	96.5%	**54.1%**	0.56	2,928	5,417	—
30	wicket	drop	30.9%	3.21	99.1%	**60.4%**	0.61	4,189	6,933	—
31	parquet-java	drop	29.1%	1.43	41.5%	**30.8%**	0.74	644	2,092	—
32	jena	drop	24.4%	5.85	142.8%	**63.0%**	0.44	1,409	2,238	—
33	dubbo	drop	11.0%	114.00	1256.4%	**61.5%**	0.05	48	78	denominator < 500, not usable; 87.79% of cited keys postdate the snapshot
34	helix	drop	10.5%	5.75	60.1%	**42.8%**	0.71	367	857	—
35	rocketmq	drop	5.7%	23.87	137.0%	**36.2%**	0.26	139	384	denominator < 500, not usable
36	pinot	drop	5.2%	1117.07	5864.3%	**7.1%**	0.00	1	14	denominator < 500, not usable; 99.84% of cited keys postdate the snapshot

Table 3: *(continued)*

#	project	bar	commit-side	commits/ ticket	ceiling	ticket realisation	TRR/ ceiling	cited ≤ N	tracker N	note
37	skywalking	drop	0.3%	—	—	—	—	0	0	no tracker record for the cited key; 100.00% of cited keys postdate the snapshot
38	shardingsphere	drop	0.0%	—	—	—	—	0	0	no tracker record for the cited key; 100.00% of cited keys postdate the snapshot

Association. Over the 33 projects with a usable denominator, Spearman ρ between the commit-side rate and the ticket realisation rate is **-0.010**. Projects clearing the bar ($n = 12$) run 0.0–68.6% (median 52.6%); projects the bar dropped ($n = 21$) run 29.5–84.8% (median 53.2%). **21 dropped projects have a higher ticket realisation rate than the worst passing one**: accumulo, atlas, calcite, cxf, flink, flume, helix, hudi, james-project, jena, karaf, oodt, parquet-java, servicecomb-java-chassis, storm, struts, syncope, tika, tomee, wicket, zeppelin.

Excluded from the association. Denominator under 500 issues: pinot, dubbo, rocketmq. No tracker record for the probed key at all: shardingsphere, skywalking — these are taxonomy modes 1 and 6 rather than low rates, and scoring them would put a number where a category belongs.

Missing clones, if any: none.